KH정보교육원

# 프로그래밍 언어 응용 A

# JAVA

# 목차

Part 1. 언어특성 활용하기

Chapter 1. 자바 기본 API ..... 4

Chapter 2. 컬렉션과 제네릭 ..... 23

Part 2. 라이브러리 활용하기

Chapter 3. 예외처리 ..... 40

Chapter 4. 입출력(IO) ..... 46

Chapter 5. 공공데이터 자바에서 활용하기 ..... 62

Appendix. 이클립스 단축키 사용법 ..... 80

# Part 1. 언어특성활용하기

# Chapter 1. 자바기본 API

## 1 java.lang 패키지

자바 API는 다양한 클래스들 제공하는데 그 중 가장 기본적인 클래스들이 java.lang 패키지에 포함되어 있다. Java 프로그래밍의 기본이 되는 패키지이므로 별도의 import 없이 사용 가능하다

### 1.1 Object 클래스

자바의 모든 클래스가 직간접적으로 상속받는 최상위 클래스

자바 모든 클래스의 기본 특징을 결정한다.

- **재정의 가능한 메소드**

① protected Object clone() throws CloneNotSupportedException()

객체를 복사하여 반환

② public boolean equals(Object obj)

객체가 동일한지 확인하여 결과 반환

③ protected void finalize() throws Throwable

가비지 컬렉션이 수행될 때 자동으로 호출

④ public final Class getClass()

객체의 실행 클래스 반환

⑤ public int hashCode()

객체의 해시 코드 값 반환

⑥ public String toString()

객체를 대표하는 문자열로 반환

- clone() : 객체의 참조(주소)값만 복사하는 얕은 복사가 아닌 참조대상값(메모리)을 복사하는 깊은 복사

- **재정의 불가능한 메소드**

동기화와 관련된 스레드 제어 메소드 들이다.

① public final void notify()

② public final void notifyAll()

③ public final void wati()

④ public final void wait(long timeout)

⑤ public final void(long timeout, int nanos)

test.clone.Point.java

```
01  package test.clone;
02
03  public class Point implements Cloneable {
04      private int posX;
05      private int posY;
06
07      public Point(int posX, int posY) {
08          this.posX = posX;
09          this.posY = posY;
10      }
11      @Override   // clone 메소드 재정의
12      public Object clone() throws CloneNotSupportedException {
13          return super.clone();
14      }
15       /*** pos X, Y Getter & Setter ***/
16      public int getPosX() {
17          return posX;
18      }
19      public void setPosX(int posX) {
20          this.posX = posX;
21      }
22      public int getPosY() {
23          return posY;
24      }
25      public void setPosY(int posY) {
26          this.posY = posY;
27      }
28  }
```

test.clone.CloneEx.java

```
01  package test.clone;
02
03  public class CloneEx {
04      public void testClone() {
05          Point p1 = new Point(10, 20);
06          Point p2 = p1; // 얕은 복사 : p1 의 주소를 복사한다
07          Point p3 = null;
08
```

```
09          try {
10              p3 = (Point) p1.clone(); // 깊은 복사 : p1 의 주소가 가리키는 값을 복사한다
11          } catch (CloneNotSupportedException e) {
12              e.printStackTrace();
13          }
14
15          // p2 는 주소를 공유하므로 영향을 받지만, p3 는 영향을 받지 않음
16          p1.setPosX(111);
17          p1.setPosY(222);
18
19          System.out.println("p1 = " + p1 + " : (" + p1.getPosX() + ", " + p1.getPosY() + ")");
20          System.out.println("p2 = " + p2 + " : (" + p2.getPosX() + ", " + p2.getPosY() + ")");
21          System.out.println("p3 = " + p3 + " : (" + p3.getPosX() + ", " + p3.getPosY() + ")");
22      }
23  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.clone.CloneEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new CloneEx().testClone();
08          // Point 객체의 참조값은 컴퓨터에 따라 다를 수 있다.
09      }
10  }
```

```
p1 = test.clone.Point@7852e922 : (111, 222)
p2 = test.clone.Point@7852e922 : (111, 222)
p3 = test.clone.Point@4e25154f : (10, 20)
```

clone() 함수를 재정의하려면 Cloneable 인터페이스를 상속받아야 한다. 그렇지 않으면 CloneNotSupportedException 예외가 발생하며 이에 따른 예외 처리구문도 필요하다.

- equals()

기본 타입 변수의 값이 같은 지 비교할 때, '==' 연산자를 이용하지만 참조 타입 변수일 경우에 '==' 연산자 사용 시 참조하고 있는 대상이 같은지를 비교하게 된다.

아래 예제를 보면, str1 과 str2 는 서로 다른 주소를 가지므로 둘을 단순 비교하면 False 이지만, equals()를 사용하면 참조 타입 변수가 가리키는 값이 참조 대상의 값과 동일한 지 비교하므로 true 를 리턴하는 것을 볼 수 있다.

test.equals.EqualsEx.java

```
01 package test.equals;
02
03 public class EqualsEx {
04 public void testEquals() {
05    String str1 = new String("Apple");
06    String str2 = new String("Apple");
07
08    System.out.println("str1==str2 : " + (str1 == str2));
09    System.out.println("str1.equals(str2) : " + str1.equals(str2));
10    }
11 }
```

test.main.Main.java

```
01 package test.main;
02
03 import test.equals.EqualsEx;
04
05 public class Main {
06    public static void main(String[] args) {
07    new EqualsEx().testEquals();
08    }
09 }
```

```
str1==str2 : false
str1.equals(str2) : true
```

- getClass()

  Object의 runtime class를 반환

  JVM에 로딩되어 있는 대상 클래스의 정보를 반환한다.

test.getClass.GetClassEx.java

```
01  package test.getClass;
02
03  import java.lang.reflect.Field;
04  import test.clone.Point; // 포인트 객체는 clone 예제의 Point Class와 동일
05
06  public class GetClass {
07      public void testGetClass() {
08          Point point = new Point(10, 20);
09
10          Class cls = point.getClass();
11
12          System.out.println("getName() : " + cls.getName()); // 객체의 클래스 이름
13          System.out.println("getSuperclass() : " + cls.getSuperclass()); // 상위클래스 이름
14
15          Field[] fields = cls.getDeclaredFields();
16          for (Field f : fields) {
17          System.out.println("Field : " + f);
18          }
19      }
20  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.getClass.GetClass;
04
05  public class Main {
06      public static void main(String[] args) {
07          new GetClass().testGetClass();
08      }
09  }
```

```
getName() : test.getclass.Point
getSuperclass() : class java.lang.Object
Field : private int test.getclass.Point.posX
Field : private int test.getclass.Point.posY
```

- hashCode()

객체가 할당 받은 메모리에 맵핑되는 유일한 정수를 해시값으로 반환

메모리 사용의 안정성을 위해 프로그램이 메모리에 직접 접근하는 것을 막고 있으며 new 연산자를 이용해 메모리를 할당할 때 메모리에 접근할 수 있는 코드인 해시 코드를 생성한다.

test.hashcode.GetHashCodeEx.java

```
01 package test.hashcode;
02
03 public class GetHashCodeEx {
04 public void testHashCode() {
05       Point point = new Point(10, 20);
06
07       System.out.println(point.hashCode());
08       }
09 }
10 class Point { // Point 객체를 선언하여 hashCode 재정의
11    int posX;
12    int posY;
13
14    public Point(int posX, int posY) {
15       this.posX = posX;
16       this.posY = posY;
17    }
18
19    @Override
20    public int hashCode() { // hashCode 메소드 재정의
21       return "point HashCode : "+super.hashCode();
22    }
23 }
```

test.main.Main.java

```
01 package test.main;
02
03 import test.hashcode.GetHashCodeEx;
04
05 public class Main {
06    public static void main(String[] args) {
07       new GetHashCodeEx().testHashCode();
08       // 실행 결과는 컴퓨터에 따라 다를 수 있다.
09    }
10 }
```

```
point HashCode : 2018699554
```

- toString()

  객체의 정보를 원하는 형태로 반환시키고자 할 때 사용

Object클래스에 정의된 toString()

```
public String toString() {
   return getClass().getName() + "@"
      + Integer.toHexString(hashCode());
}
```

toString()을 오버로딩해서 사용하지 않으면 클래스 이름과 16진수 hasCode를 반환

test.toString.ToStringEx.java

```
01  package test.toString;
02
03  public class ToStringEx {
04     public void testToString() {
05        Point point = new Point(10, 20);
06
07        System.out.println(point);
08        // System.out.println(point.toString());
09     }
10  }
11
12  class Point {
13     int posX;
14     int posY;
15
16     public Point(int posX, int posY) {
17        this.posX = posX;
18        this.posY = posY;
19     }
20
21     @Override
22     public String toString() {  // toString 메소드 재정의
23        return "point 객체 : (" + posX + ", " + posY + ")";
       }
    }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.toString.ToStringEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new ToStringEx().testToString();
08      }
09  }
```

```
point 객체 : (10, 20)
```

## 1.2 String 클래스

문자열을 사용하는데 가장 많이 사용하는 클래스

String 객체가 생성되면 그 값이 길어지거나 줄어들 수 없으며, 구성 문자들 중 어떤 것도 바꿀 수 없다. 즉 우리가 String 객체의 값을 바꾼다면, 본래 객체 외의 새로운 객체를 생성하여 그 참조 주소를 가리키는 변수가 된다는 것이다.

| 함수 | 반환형 | 설명 |
|---|---|---|
| charAt(int index) | char | index 위치에 있는 문자 반환 |
| compareTo(String str) | int | str보다 사전 순으로 앞이면 음수, 같으면 0, 뒤면 정수 값을 반환 |
| concat(String str) | String | 기존 문자열에 str을 이어 새로운 String을 반환 |
| equals(String str) | boolean | str과 같은 문자열이면(대소문자 구분) True, 그렇지 않으면 False 반환 |
| equalsIgnoreCase(String str) | boolean | str과 같은 문자열이면(대소문자 무시) True, 그렇지 않으면 False 반환 |
| length() | int | 문자 길이를 반환 |
| replace(char oldChar, char newChar) | String | 문자열의 oldChar를 모두 newChar로 변경 |
| substring(int offset, int endIndex) | String | offset 위치에서부터 endIndex-1까지에 걸친 문자열을 반환 |
| toLowerCase() | String | 모든 대문자를 소문자로 변환 |
| toUpperCase() | String | 모든 소문자를 대문자로 변환 |

- String Class 예제

test.str.StringEx.java

```
01  package test.str;
02
03  public class StringEx {
04      public void testString() {
05          String str = "Hello Java";    // 공백도 하나의 문자로 취급
06          String concat, upperCase, replace, subString;
07
08          System.out.println("기본 String : "+ str);
09          System.out.println("기본 String 길이 : "+str.length());     // 문자열 길이 출력
10
11          concat = str.concat(", Hi Java");       // 문자열 연결
12          upperCase = concat.toUpperCase();              // 문자열 대문자로 변환
```

```
13          replace = upperCase.replace('J', 'L');//  문자 J를 L로 변환
14          subString = replace.substring(3, 10);            // 3~9번째 위치의 문자열 잘라내기
15
16          System.out.println("Concat String : " + concat);
17          System.out.println("upperCase String : " + upperCase);
18          System.out.println("replace String : " + replace);
19          System.out.println("subString String : " + subString);
20      }
21  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.str.StringEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new StringEx().testString();
08      }
09  }
```

```
기본 String : Hello Java
기본 String 길이 : 10
Concat String : Hello Java, Hi Java
upperCase String : HELLO JAVA, HI JAVA
replace String : HELLO LAVA, HI LAVA
subString String : LO LAVA
```

## 1.3 StringBuffer 클래스와 StringBuilder 클래스

문자열을 다룰 때 사용하며, 문자열을 변경할 때마다 새로운 객체를 생성하는 String 클래스와 다르게 문자열 변경 시 새로운 객체를 생성하지 않고, 버퍼에 담겨 있는 문자열을 바로 수정한다.

StringBuffer 클래스와 StringBuilder 클래스의 객체는 객체 생성 시 크기를 지정하지 않으면 기본적으로 16개의 문자를 저장할 수 있는 버퍼를 가진다. StringBuffer 클래스와 StringBuilder 클래스는 기본적인 동작은 같으며 StringBuffer 클래스만 동기화(Thread Safe)를 지원한다는 점이 다르다

- StringBuffer Class 예제

test.str.StringBufferEx.java

```
01  package test.str;
02
03  public class StringBufferEx {
04      public void testStringBuffer() {
05              StringBuffer str1 = new StringBuffer();
06              StringBuffer str2 = new StringBuffer(30);
07              StringBuffer str3 = new StringBuffer("Java");
08
09              str1.append("HI");
10              str2.append("Hello");
11              str3.insert(0, str2 + " ");
12
13              System.out.println(str1);
14              System.out.println(str2);
15              System.out.println(str3);
16      }
17  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.str.StringBufferEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          new StringBufferEx().testStringBuffer();
08      }
09  }
```

```
HI
Hello
Hello Java
```

# 1.4 Math 클래스

숫자 계산에 흔히 사용되는 수학적 기능을 가지는 메소드를 제공하는 클래스

모든 메소드들이 정적 메소드로 정의되어 있어 객체를 생성하지 않고 사용한다

| 메소드 | 설명 |
|---|---|
| static int abs(int num) | 절대 값을 반환 |
| static double acos(double num) | arc cosine을 반환 |
| static double asin(double num) | arc sine을 반환 |
| static double atan(double num) | arc tangent를 반환 |
| static double cos(double num) | cosine을 반환 |
| static double sin(double num) | sine을 반환 |
| static double tan(double tan) | tangent를 반환 |
| static double ceil(double num) | 올림 값(ceiling)을 반환 |
| static double floor(double num) | 내림 값을 반환 |
| static double exp(double power) | 지정된 power만큼 e의 제곱 된 값을 반환 |
| static double pow(double num, double power) | num을 power 만큼 제곱한 값을 반환 |
| static double random() | 0.0(포함)과 1.0(미포함) 사이의 난수를 반환 |
| static double sqrt(double num) | 양수만 가능한 num의 제곱근을 반환 |

- Math Class 예제

test.math.MathTest.java

```
01  package test.math;
02
03  public class MathTest {
04      public void printMath() {
05          System.out.println("abs(-10) : "+Math.abs(-10));          // -10의 절대값
06          System.out.println("ceil(3.7) : "+Math.ceil(3.7));        // 3.7의 올림
07          System.out.println("floor(3.7) : "+Math.floor(3.7));      // 3.7의 내림
08          System.out.println("exp(3.3) : "+Math.exp(3.3));          // e^3.3
09          System.out.println("pow(2.1,2) : "+Math.pow(2.1, 2));     // 2.1^2
10          System.out.println("random() : "+Math.random());          // 0.0 ~ 1.0 사이의 난수
11      }
12  }
```

| | |
|---|---|
| | |

test.main.Main.java

```
01  package test.main;
02
03  import test.math.MathTest;
04
05  public class Main {
06      public static void main(String[] args) {
07          new MathTest().printMath();
08      }
09  }
```

```
abs(-10) : 10
ceil(3.7) : 4.0
floor(3.7) : 3.0
exp(3.3) : 27.112638920657883
pow(2.1,2) : 4.41
random() : 0.10559212646066529
```

# 1.5 Wrapper 클래스

자바의 기본형 데이터타입을 클래스화하기 위해 사용하는 클래스

기본형 데이터타입을 클래스화하면 Object 클래스의 자식 클래스가 되며 데이터타입에 상관없이 Object클래스를 이용한 처리가 가능해진다

| 기본형 데이터타입 | Wrapper 클래스 |
|---|---|
| boolean | Boolean |
| byte | Byte |
| char | Character |
| short | Short |
| int | Integer |
| long | Long |
| float | Float |
| double | Double |

- Object형 매개변수를 이용하여 하나의 함수로 int형과 double형 데이터 처리

test.wrapper.WrapperEx.java

```
01  package test.wrapper;
02
03  public class WrapperEx {
04      public void printWrapper(Object obj) {
05          System.out.println("데이터 : " + obj);
06          System.out.println("데이터 클래스 : " + obj.getClass());
07      }
08  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.wrapper.WrapperEx;
04
05  public class Main {
06      public static void main(String[] args) {
07          int i = 123;
08          double d = 3.14;
09          WrapperEx wp = new WrapperEx();
10          wp.printWrapper(i);
```

```
11          wp.printWrapper(d);
12      }
13  }
```

```
데이터 : 123
데이터 클래스 : class.java.lang.Integer
데이터 : 3.14
데이터 클래스 : class.java.lang.Double
```

## 1.6 박싱(Boxing), 언박싱(Unboxing)

### 1.6.1 박싱(Boxing)

기본형 데이터타입을 객체화 시키는 작업

```
int i = 10;
Integer iValue1 = new Integer(i);
Integer iValue2 = new Integer(123);

double d = 1.123;
Double dValue1 = new Double(d);
Double dValue2 = new Double(5.323);
```

### 1.6.2 언박싱(Unboxing)

언박싱(Unboxing) : 객체에 저장되어 있는 데이터를 기본형 데이터로 꺼내는 작업

```
Integer iValue = new Integer(4578);
int i = iValue.intValue();

Double dValue = new Double(44.241);
double d = dValue.doubleValue();
```

## 1.7 자동 박싱(Auto Boxing), 자동 언박싱(Auto Unboxing)

자바 JDK1.5 이상부터는 기본 타입과 Wrapper클래스 타입 간의 변환이 자동으로 이루어진다

```
int i = 10;
Integer iValue = i;

Double dValue = new Double(3.14);
double d = dValue;
```

# 2 그 외 유용한 클래스

## 2.1 Objects 클래스

Object 클래스와 유사한 이름을 가진 클래스로 객체 비교, 해시코드 생성, null 여부, 객체 문자열 리턴 등의 연산을 수행할 수 있다.

| 메소드 | 설명 |
|---|---|
| int compare(T a, Tb, Comparator<T> c) | 두 객체 a와 b를 Comparator를 사용해 비교 |
| int hashCode(Object o) | 객체의 해시코드 생성 |
| Boolean isNull(Ojbect obj) | 널인지 확인 |
| Boolean nonNull(Ojbect obj) | 널이 아닌지 확인 |
| String toString(Object o) | 객체의 toString()값 리턴 |

- Comparator를 이용하여 Student 객체의 총점을 비교하는 예제

test.vo.Student.java

```
01 package test.vo;
02
03 public class Student {
04     private String name;
05     private int kor;
06     private int eng;
07
08     public Student(String name, int kor, int eng) {
09         this.name = name;
10         this.kor = kor;
11         this.eng = eng;
12     }
13     public int getSum() { return kor+eng; }
14     public String getName() { return name; }
15 }
```

test.controller.StuComparator.java

```
01 package test.controller;
02
03 import java.util.Comparator;
04 import test.vo.Student;
05
```

```
06  public class StuComparator implements Comparator<Student>{
07      @Override  // Comparator 인터페이스의 compare() 메소드를 재정의
08      public int compare(Student o1, Student o2) {
09          if(o1.getSum() > o2.getSum()) return 1;
10          else if(o1.getSum() < o2.getSum()) return -1;
11          else return 0;
12      }
13  }
```

test.controller.StuManager.java

```
01  package test.controller;
02
03  import java.util.Objects;
04  import test.vo.Student;
05
06  public class StuManager {
07      public void scoreComp(Student s1, Student s2) {
08          new StuComparator().compare(s1, s2);
09          int result = Objects.compare(s1, s2, new StuComparator());
10
11          if(result > 0 )    System.out.println(s1.getName() + " 학생이 우수합니다");
12          else if(result < 0 )    System.out.println(s2.getName() + " 학생이 우수합니다");
13          else    System.out.println("두 학생은 점수가 같습니다");
14      }
15  }
```

test.main.Main.java

```
01  package test.view;
02
03  import test.controller.StuManager;
04  import test.vo.Student;
05
06  public class Main {
07      public static void main(String[] args) {
08          Student s1 = new Student("홍길동", 100, 90);
09          Student s2 = new Student("고길동", 90, 80);
10          new StuManager().scoreComp(s1, s2);
11      }
12  }
```

```
홍길동 학생이 우수합니다
```

## 2.2 Random 클래스

자바에서 난수를 발생시키는 방법은 두 가지가 있다. Math 클래스의 random() 메소드를 이용하는 방법과 Random 클래스를 이용하는 방법이다.

Math.random()은 정적 메소드로 객체를 생성하지 않고 편하게 사용할 수 있지만 double형태의 난수만 발생시킬 수 있다. Random클래스는 객체를 생성해야하는 불편함이 있지만 int, long float, double, boolean형의 난수를 발생시킬 수 있다

- Random 클래스의 메소드

| 메소드 | 설명 |
|---|---|
| int nextInt() | int 형 범위 안의 난수 발생 |
| int nextInt(int n) | 0부터 정수 n까지의 난수 발생 |
| boolean nextBoolean() | 무작위로 true 또는 false 반환 |
| float nextFloat() | float 범위의 난수 발생 |
| double nextDouble() | double 범위의 난수 발생 |

test.random.RandomEx.java

```
01  package test.random;
02
03  public class RandomEx {
04      public void testRandom() {
05          Random r = new Random();
06
07          System.out.println("nextInt() : "+ r.nextInt());            // -2^31 ~ 2^31-1 사이의 난수
08          System.out.println("nextInt(100) : "+ r.nextInt(100));  // 0~100 사이의 난수
09          System.out.println("nextBoolean() : "+ r.nextBoolean());    // true 또는 false
10          System.out.println("nextDouble() : "+ r.nextDouble());    // double형 표현범위의 난수
11          System.out.println("nextFloat() : "+ r.nextFloat());       // float형 표현범위의 난수
12      }
13  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.random.RandomEx;
04
05  public class Main {
06      public static void main(String[] args) {
```

```
07          new RandomEx().testRandom();
08      }
09  }
```

```
nextInt() : -2014602294
nextInt(100) : 65
nextBoolean() : true
nextDouble() : 0.7777749855581843
nextFloat() : 0.16439652
```

# Chapter 2. 컬렉션과 제네릭

## 1 컬렉션 프레임워크

객체를 그룹 단위의 데이터로 저장하는 기본적인 자료구조들의 모음

다수의 데이터를 그룹으로 묶어 관리할 수 있으므로 프로그래밍이 보다 편리해지고, 코드의 재사용성을 늘릴 수 있다.

### 1.1 컬렉션 프레임워크 종류

컬렉션 프레임워크의 주요 인터페이스는 List, Set, Map이 있다

다음 표는 컬렉션의 종류에 따른 차이를 표로 정리한 것이다.

| 인터페이스 분류 | | 특징 | 구현 클래스 |
|---|---|---|---|
| Collection | List | 순서를 유지하고 저장<br>중복 저장 가능 | ArrayList, Vector<br>LinkedList |
| | Set | 순서를 유지하지 않고 저장<br>중복 저장 안됨 | HashSet, TreeSet |
| Map | | 키와 값의 쌍으로 저장<br>키는 중복 안 됨<br>값은 중복 저장 가능 | HashMap, Hashtable<br>TreeMap, LinkedHashMap |

## 2 List

객체를 일렬로 늘어놓은 구조

객체를 인덱스로 관리

객체를 저장하면 자동 인덱스가 부여되고 인덱스로 객체를 검색, 삭제할 수 있는 기능을 제공

### 2.1 List 인터페이스로 구현된 클래스

① ArrayList

② Vector

③ LinkedList

## 2.2 List 인터페이스에 구현된 메소드

| 기능 | 메소드 | 설명 |
|---|---|---|
| 객체추가 | boolean add(E e) | 주어진 객체를 맨 끝에 추가 |
| | void add(int index, E element) | 주어진 인덱스에 객체 추가 |
| | set(int index, E element) | 주어진 인덱스에 저장된 객체를 주어진 객체로 바꿈 |
| 객체 검색 | boolean contains(Object o) | 주어진 객체가 저장되어 있는지 여부 확인 |
| | E get(int index) | 주어진 인덱스에 저장된 객체를 리턴 |
| | boolean isEmpty() | 컬렉션이 비어 있는지 조사 |
| | int size() | 저장되어 있는 전체 객체 수를 리턴 |
| 객체 삭제 | void clear() | 저장된 모든 객체를 삭제 |
| | E remove(int index) | 주어진 인덱스에 저장된 객체를 삭제 |
| | boolean remove(Object o) | 주어진 객체를 삭제 |

## 2.3 ArrayList – 가변 배열

일반 배열과 같이 인덱스로 객체를 관리

배열은 생성될 때 크기가 고정되어 변경을 할 수 없지만 ArrayList는 저장 용량을 초과하면 자동으로 저장 용량을 늘림

상당히 빠르고 크기를 마음대로 조절할 수 있는 배열

단방향 포인터 구조로 자료에 대한 순차적이 접근에 강점이 있다

- 기본 생성자로 ArrayList 객체를 생성

```
List<기본타입> list = new ArrayList<기본타입>();
```

- 초기화

고정된 객체들로 구성된 List를 생성하고자 할 때 사용

```
List<String> list = Arrays.asList("Apple", "Banana", "Orange");
```

test.collection.TestArrayList.java

```
01 package test.collection;
02
03 import java.util.*;
04
05 public class TestArrayList {
06     private List<String> l = new ArrayList<String>();
07
08     public void testArrayList() {
09         l.add("Red");                    //[Red]
```

```
10          l.add("Orange");                    //[Red, Orange]
11          l.add(1, "Yellow");         //[Red, Yellow, Orange]
12          printList();
13
14          l.remove(2);                //[Red, Yellow]
15          printList();
16
17          l.clear();                  //[ ]
18          System.out.println("List가 비었나요 ? "+l.isEmpty());      //true
19      }
20
21      public void printList() {
22          System.out.println(l);
23      }
24  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.collection.TestArrayList;
04
05  public class Main {
06      public static void main(String[] args) {
07          new TestArrayList().testArrayList();
08      }
09  }
```

```
[Red, Yellow, Orange]
[Red, Yellow]
List가 비었나요 ? true
```

## 2.4 Vector

ArrayList의 구 버전

ArrayList와 동일한 내부 구조를 가지고 있다

모든 메소드가 동기화 되어있고 잘 쓰이지 않음

- 기본 생성자로 Vector 객체를 생성

```
List<E> list = new Vector<E>();
```

ArrayList와는 달리 동기화된 메소드로 구성되어 있어 멀티 스레드환경에서 동시에 메소드들을 실행 할 수 없고, 하나의 스레드의 실행이 완료되어야만 다른 스레드에서 실행할 수 있다.

## 2.5 LinkedList

양방향 포인터 구조로 데이터의 삽입, 삭제가 빈번할 경우 빠른 성능을 보장

ArrayList와 사용 방법은 똑같지만 내부 구현이 인접 참조 링크를 체인처럼 관리

ArrayList는 객체가 중간 인덱스에 삭제되거나 삽입될 때 뒤의 객체들의 인덱스가 변경되지만,

LinkedList는 중간 인덱스의 객체가 변경되더라도 앞뒤 링크만 변경되고 나머지 링크는 변경되지 않음

삭제와 삽입이 빈번이 일어나는 경우 ArrayList보다 LinkedlList가 좋은 성능을 발휘한다.

스택, 큐, 양방향 큐등을 만들기 위한 용도로 쓰임

```
List<E> list = new LinkedList<E>();
```

test.collection.TestLinkedList.java

```
01  package test.collection;
02
03  import java.util.LinkedList;
04
05  public class TestLinkedList {
06    public TestLinkedList() {
07    }
08
09    public void testLinkedList() {
10      LinkedList<String> list = new LinkedList<String>();
11
12      list.add("포도"); // [포도]
13      list.add("딸기"); // [포도, 딸기]
14      list.add("복숭아"); // [포도, 딸기, 복숭아]
15      list.add("키위"); // [포도, 딸기, 복숭아, 키위]
16      printList(list);
17
18      list.set(0, "오렌지"); // [오렌지, 딸기, 복숭아] -> set으로 0번째 인덱스 변경
19      printList(list);
20
21      list.remove(1); // [오렌지, 딸기, 복숭아] -> remove로 1번째 인덱스 삭제
22      list.remove("키위"); // "키위" 데이터 삭제
23      printList(list);
24    }
25
26    public void printList(LinkedList<String> list) {
```

```
27      // 반복문을 이용한 LinkedList 출력
28      int num = list.size();
29      for (int cnt = 0; cnt < num; cnt++) {
30        String str = "["+list.get(cnt)+"] ";
31        System.out.print(str);
32      }
33      System.out.println();
34    }
35  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.collection.TestLinkedList;
04
05  public class Main {
06    public static void main(String[] args) {
07      new TestLinkedList().testLinkedList();
08    }
09  }
```

```
[포도] [딸기] [복숭아] [키위]
[오렌지] [딸기] [복숭아] [키위]
[오렌지] [복숭아]
```

# 3 Set

저장 순서가 유지되지 않으며, 객체를 중복해서 저장할 수 없음

인덱스로 관리하지 않기 때문에 인덱스를 매개 변수로 갖는 메소드가 없음

## 3.1 Set 인터페이스로 구현된 클래스

① HashSet

② LinkedHashSet

③ TreeSet

## 3.2 Set 인터페이스에 구현된 메소드

| 기능 | 메소드 | 설명 |
|---|---|---|
| 객체추가 | boolean add(E e) | 주어진 객체 e 를 추가 |
| 객체 검색 | boolean contains(Object o) | 주어진 객체가 저장되어 있는지 여부 확인 |
| | boolean isEmpty() | 컬렉션이 비어 있는지 조사 |
| | int size() | 저장되어 있는 전체 객체 수를 리턴 |
| | Object[] toArray() | 배열로 반환 |
| | Object[] toArray(Object[] a) | 지정한 배열의 클래스 타입으로 배열로 반환 |
| 객체 삭제 | void clear() | 저장된 모든 객체를 삭제 |
| | boolean remove(Object o) | 주어진 객체를 삭제 |
| | boolean removeAll(Collection c) | 저장된 모든 객체를 삭제 |

## 3.3 HashSet

내부적으로 HashMap을 사용

객체들을 순서 없이 저장하고 동일한 객체는 중복 저장하지 않음

저장하려는 객체의 해시코드와 저장된 객체들의 해시코드를 비교해 같은 값이 없다면 객체 추가

```
Set<E> set = new HashSet<E>();
```

## 3.4 LinkedHashSet

해시 테이블에 데이터를 저장

저장된 순서에 따라 순서가 결정된다.

추가된 순서, 또는 가장 최근에 접근한 순서대로 접근 가능

test.collection.TestLinkedHashSet.java

```
01  package test.collection;
02
03  import java.util.LinkedHashSet;
04
05  public class TestLinkedHashSet {
06
07    public void testLinkedHashSet() {
08      LinkedHashSet<String> set = new LinkedHashSet<String>();
09
10      set.add("Java");
11      set.add("Oracle");
12      set.add("JDBC");
13      set.add("HTML5");
14
15      for(String text : set) {  // HashSet 과 다르게 값이 들어간 순서대로 출력된다.
16        System.out.println(text);
17      }
18    }
19  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.collection.TestLinkedHashSet;
04
05  public class Main {
06        public static void main(String[] args) {
07          new TestLinkedHashSet().testLinkedHashSet();
08        }
09  }
```

```
Java
Oracle
JDBC
HTML5
```

### 3.5 TreeSet

정렬기능이 추가되었고 동일한 객체는 중복 저장하지 않음

정렬된 순서대로 보관하며 정렬 방법을 지정할 수 있음

범위 검색 작업에 효과적

레드블랙트리 구조를 사용

HashSet보다 성능상 느리지만 삽입과 동시에 정렬할 때 사용.

test.collection.TestTreeSet.java

```
01  package test.collection;
02
03  import java.util.TreeSet;
04
05  public class TestTreeSet {
06      public void testTreeSet() {
07          TreeSet<String> tset = new TreeSet<String>();
08
09          tset.add("3. Mybatis");
10          tset.add("1. Java");
11          tset.add("5. HTML5");
12          tset.add("4. Oracle");
13          tset.add("2. JDBC")
14
15          for(String text : tset) {  // 앞의 번호 순으로 자동 정렬되어 출력된다.
16              System.out.println(text);
17          }
18      }
19  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.collection.TestTreeSet;
04
05  public class Main {
06      public static void main(String[] args) {
07          new TestTreeSet().testTreeSet();
08      }
09  }
```

```
1. Java
```

2. JDBC
3. Mybatis
4. Oracle
5. HTML5

# 4 Map

키와 값으로 구성된 엔트리 객체를 저장하며, 키와 값 모두 객체로, 키는 중복될 수 없지만 값은 중복 가능하다. 기존에 저장된 키로 값을 저장하면 덮어씌워진다.

## 4.1 Map 인터페이스로 구현된 클래스

① HashMap
② Hashtable
③ Properties

## 4.2 Map 인터페이스에 구현된 메소드

| 기능 | 메소드 | 설명 |
|---|---|---|
| 객체추가 | V put(K key, V value) | 주어진 키와 값을 추가 |
| 객체 검색 | V get(Object key) | 주어진 키의 값을 리턴 |
| | boolean contains(Object key) | 주어진 키가 있는지 여부 |
| | boolean containsValue(Object value) | 주어진 값이 있는지 여부 |
| | Set<Map.Entry<K,V>> entrySet() | 주어진 키와 값의 쌍으로 구성된 모든 엔트리 객체를 Set에 담아 리턴 |
| | boolean isEmpty() | 컬렉션이 비어 있는지 조사 |
| | int size() | 저장되어 있는 전체 객체 수를 리턴 |
| | Set<K> keySet() | 모든 키를 Set 객체에 담아 리턴 |
| | Collenction<V> values() | 모든 값을 Collection 객체에 담아 리턴 |
| 객체 삭제 | void clear() | 저장된 엔트리객체를 삭제 |
| | V remove(Object key) | 주어진 키와 일치하는 엔트리객체를 삭제 |

## 4.3 HashMap

Map 인터페이스를 구현한 대표 Map 컬렉션
키는 중복 저장을 허용하지 않으며, 값은 중복을 허용
동일한 키가 될 조건은 hashCode()의 리턴값이 같고, equals() 메소드의 결과가 true이어야 함

## 4.4 Hashtable

HashMap과 동일한 내부 구조를 가지고 있음
HashMap과 달리 동기화된 메소드로 구성되어 있어 멀티 스레드 환경에서 동시에 구현된 메소드들을 실행할 수 없다.

test.map.TestHashMap.java

```
01  package test.map;
02
03  import java.util.HashMap;
04  import java.util.Iterator;
05
06  public class TestHashMap {
07      public void testMap() {
08          HashMap<Integer,String> map = new HashMap<Integer,String>();
09          map.put(1, "Apple");
10          map.put(4, "Dandelion");
11          map.put(3, "Corn");
12          map.put(2, "BlackBerry");
13
14          Iterator<Integer> iterKey = map.keySet().iterator();
15
16          while(iterKey.hasNext()) {  // 작성한 map 객체를 목록화하여 각각 출력한다
17              int key = iterKey.next();
18              System.out.println("key "+key+" : "+map.get(key));
19          }
20      }
21  }
```

test.main.Main.java

```
01  package test.main;
02
03  import test.map.TestHashMap;
04
05  public class Main {
06      public static void main(String[] args) {
07          new TestHashMap().testMap();
08      }
09  }
```

```
key 1 : Apple
key 2 : BlackBerry
key 3 : Corn
key 4 : Dandelion
```

## 4.5 Properties

Hashtable의 하위 클래스로, 키와 값을 String 타입으로 제한

애플리케이션의 옵션 정보, 데이터베이스 연결 정보, 다국어 정보가 저장된 프로퍼티(*.properties) 파일을 읽을 때 주로 사용하며, 예제에 관련 IO 클래스를 포함하여 작성하였다. IO 클래스는 이후 입출력 파트에서 자세히 다루도록 하겠다.

test.map.PropertiesStudent.java

```
01  package test.map;
02
03  public class PropertiesStudent implements java.io.Serializable {
04      private String no;
05      private String name;
06      private int score;
07
08      public Student() {}
09      public Student(String no, String name, int score) {
10      this.no = no;
11      this.name = name;
12      this.score = score;
13      }
14      @Override
15      public String toString() {
16      return no+", "+name+", "+score;
17      }
18      /*... Getter & Setter 작성부 ... */
19  }
```

test.map.StudentManager.java

```
01  package test.map;
02
03  import java.io.*;
04  import java.util.*;
05
06  public class StudentManager {
07      private ArrayList<Student> list = new ArrayList<Student>();
08      private Properties prop = new Properties();
09
10      public StudentManager() {}
11      public void propertyInsert(Student st) {
12          prop.setProperty(st.getNo(),st.toString());
```

```
13          System.out.println(st.getName()+" 학생의 데이터 삽입 성공!");
14      }
15      public void propertyStore() {
16          try { // 설정 파일 저장 시....
17              prop.store(new FileWriter(new File("prop.properties")), "Ex_Student.properties");
18              System.out.println("학생 데이터 저장 성공!");
19          } catch (Exception e) {
20              e.printStackTrace();
21          }
22      }
23      public void propertyPrint() {
24          try { // 설정 파일 불러올 시....
25              prop.load(new FileReader(new File("prop.properties")));
26              for(int i = 1; i <= prop.size(); i++) {
27                  String index = String.valueOf(i);  // String 형변환을 통한 key 값 도출
28                  list.add(parseStudent(prop.getProperty(index)));
29              }
30          } catch (Exception e) {
31              e.printStackTrace();
32          }
33
34          for(Student s : list) {
35              System.out.println(s);
36          }
37          System.out.println("학생 데이터 출력이 완료되었습니다.");
38      }
39
40      public Student parseStudent(String str) { // 넘겨 받은 데이터를 학생 객체로 변환
41          String tmp[] = str.split(", ");
42          return new Student(tmp[0], tmp[1], Integer.parseInt(tmp[2]));
43      }
44  }
```

test.main.PropertiesMain.java

```
01  package test.main;
02
03  import test.map.PropertiesStudent;
04  import test.map.StudentManager;
05
06  public class PropertiesMain {
```

```
07    public static void main(String[] args) {
08        StudentManager stm = new StudentManager();
09
10    // properties 데이터 삽입
11    stm.propertyInsert(new Student("1","홍길동",80));
12    stm.propertyInsert(new Student("2","김유신",60));
13    stm.propertyInsert(new Student("3","신사임당",95));
14
15    // properties 데이터 저장
16    stm.propertyStore();
17
18    // properties 데이터 입출력
19    stm.propertyPrint();
20    }
21 }
```

prop.properties

```
01 #Ex_Student.properties
02 1=1, 홍길동, 80
03 2=2, 김유신, 60
04 3=3, 신사임당, 95
```

# 5 Stack과 Queue

Stack : 후입선출(LIFO, Last In First Out) 자료구조

Queue : 선입선출(FIFO, First In Firt Out) 자료구조

## 5.1 Stack

LIFO 자료구조를 구현한 클래스

- Stack 객체 생성

```
Stack<E> stack = new Stack<E>();
```

- Stack 클래스 주요 메소드

| 리턴 타입 | 메소드 | 설명 |
|---|---|---|
| E | push(E item) | 주어진 객체를 스택에 추가 |
| E | peek(0 | 스택의 최상단(top) 객체를 가져온다. 객체를 제거하지 않음 |
| E | pop() | 스택의 최상단(top) 객체를 가져온다. 가져온 객체를 제거함 |

### 5.2 Queue

FIFO 자료구조를 구현한 클래스

- Queue 객체 생성

```
Queue<E> queue = new Queue<E>();
```

- Queue 클래스 주요 메소드

| 리턴 타입 | 메소드 | 설명 |
|---|---|---|
| boolean | offer(E e) | 주어진 객체를 추가 |
| E | peek(0 | 가장 처음 추가된 객체(front)를 가져온다. 객체를 제거하지 않음 |
| E | pop() | 가장 처음 추가된 객체(front)를 가져온다. 가져온 객체를 제거 |

## 6 동기화된 컬렉션

대부분의 컬렉션 클래스들은 싱글 스레드 환경에서 사용할 수 있도록 설계

멀티 스레드환경에서 무결성을 보장하지 못 함

Vector 와 Hashtalbe은 동기화된 메소드로 구성되어 있어 멀티 스레드 환경에서 안전하게 사용 가능

싱글 스레드 환경에 맞게 설계된 컬렉션 객체들을 동기화된 메소드로 래핑할 수 있도록 Collections가 메소드들을 제공

- Collections의 동기화 래핑 메소드

| 리턴타입 | 메소드 | 설명 |
|---|---|---|
| List<T> | synchronizedList(List<T> list) | List를 동기화된 List로 리턴 |
| Map<K, V> | synchronizedMap(Map<K, V> m) | Map을 동기화된 Map으로 리턴 |
| Set<T> | synchronizedSet(Set<T> s) | Set을 동기화된 Set으로 리턴 |

- Collections의 동기화 객체 생성

```
List<T> syncList = Collections.synchronizedList(list);
Map<K, V> syncMap = Collections.synchronizedMap(map);
Set<T> syncSet = Collections.synchronizedSet(set);
```

# 7 제네릭

다양한 타입의 객체들을 다루는 메소드나 클래스의 타입을 미리 명시해 줌으로써 컴파일 시 타입을 체크할 수 있도록 하고, 형변환을 하는 번거로움 없이 사용하게 해주는 것으로 배열이나 이후 배울 컬렉션 사용 시 데이터 타입의 안정성을 높일 수 있다

* 타입 안정성

의도하지 않는 타입의 객체가 저장되는 것을 막고, 저장된 객체를 의도하지 않은 타입으로 사용되는 것을 막는 것

# 8 제네릭 선언

제네릭은 클래스와 메소드에 선언 가능

- 제네릭 클래스 선언

```
[접근제한자] class [클래스이름] <Type1, Type2, ... > { }
```

클래스를 선언할 때 클래스 이름 옆에 < >를 이용하여 데이터 타입이나 타입 변수를 지정해주면 된다. 제네릭 타입 변수는 식별자 명명 규칙에 따라 어느 것이든 가능하지만 일반적으로 T를 많이 사용한다. 이후 클래스 내부에 변수나 메소드를 선언할 때 일반 데이터 타입과 동일하게 사용하면 되며, 타입 변수를 특정 데이터 타입으로 지정해주면 해당 타입으로 메모리에 할당하여 사용 가능하다.

```
public class GenericEx<T> {
   T object;

   void setObject(T object) { this.object = object; }
   T getObject() { return object; }
}

GenericEx<String> ex = new GenericEx<String>();
```

만약 두 개 이상의 타입으로 제네릭 타입 변수를 사용하고 싶다면 콤마로 구분하여 다른 이름으로 타입 변수를 지정해주면 된다.

# 9 제네릭 메소드

메소드에서 사용할 타입을 제네릭을 이용하여 선언할 수 있다.

```
[접근제한자] <Type1, Type2, ... > [리턴타입] [메소드이름](매개변수, ...)  { ... }
```

제네릭 메소드를 호출할 때에는 타입 변수를 명시적으로 지정할 수도 있고 컴파일러가 매개값의 타입을 보고 추정하도록 할 수도 있다.

## 10 제네릭 제한하기

일반적인 방법으로 제네릭을 이용했을 경우 타입에 대한 제한이 없다. 하지만 제네릭 타입 변수에 extends 키워드를 사용하면 타입의 종류를 제한할 수 있다.

```
class Point {
   int x;
   int y;
}

class Triangle<T extends Point> {
   T pos1, pos2, pos3;
}
```

extends 뒤에 명시된 타입의 자손들만 타입 변수에 대입할 수 있게 된다.

## 11 와일드카드

타입 변수를 매개변수나 리턴 타입으로 사용할 때 구체적인 타입 대신 와일드 카드를 사용할 수 있다. 타입 변수를 대치할 수 있는 구체적인 타입으로 제한을 두지 않거나 extends 키워드를 이용해 해당 타입과 자손 타입들만 가능하도록 제한하거나 super키워드를 이용해 해당 타입과 부모 타입들만 가능하도록 제한하는 방법이 있다.

- 와일드카드 이용 3가지 방법

```
<?> : 제한 없음
<? extends 상위타입> : 상위 클래스 제한
<? super 하위 타입> : 하위 클래스 제한
```

## 12 제네릭 상속

제네릭 클래스도 다른 클래스와 마찬가지로 부모 클래스가 될 수 있다.

부모 클래스의 제네릭 타입을 상속해서 자식 클래스에서도 제네릭 타입을 상속할 수 있다.

```
public class ChildProduct<T, K> extends Product<T, K> { ... }
```

자식 제네릭 클래스는 추가적인 타입 변수를 가질 수 있다.

```
public class ChildProduct<T, K, S> extends Product<T, K> { ... }
```

# Chapter 3. 예외 처리

## 1 예외 (Exception)

프로그래밍 오류(에러)는 발생 시점에 따라 컴파일타임 에러, 런타임 에러, 시스템 에러 세가지로 구분할 수 있다.

- **컴파일 에러 :** 소스 컴파일 시에 발생하며. 주로 문법적인 에러이다. 에러가 발생된 소스 코드를 수정하여야 한다.
- **런타임 에러 :** 프로그램 실행 도중에 발생하며. 잘못된 값의 입력 등 수행할 수 없는 작업을 시도할 경우에 발생한다. 소스 코드로 에러 발생 구문을 수정하여야 한다.
- **시스템 에러 :** 컴퓨터 시스템 상에 발생하는 오동작에 의한 에러로 정정, 메모리 부족 등 소스 코드로는 해결할 수 없는 장비 또는 운영체제 관련 에러이다. 소스 코드 수정으로는 해결할 수 없다.

자바는 런타임 에러를 '에러(error)'와 '예외(exception)'으로 구분한다. 에러는 프로그램 소스 코드로 해결할 수 없는 경우를 말하며 복구할 수 없는 심각한 경우이다. 예외는 소스 코드로 해결할 수 있는 에러를 말하며 발생하더라도 해결할 수 있다.
자바는 예외는 크게 두가지로 구분하였는데, RuntimeException이나 그 후손들은 Unchekced Exception 이라 하여 소스 코드상에서 처리하지 않아도 되는 예외로, 그 외의 예외들을 Checked Exception 이라 하여 반드시 소스 코드로 예외 처리(Exception Handling)를 해주어야 하는 예외로 구분해 두었다

## 2 예외 처리 (Exception Handling) 구문

예외 처리 구문은 크게 세 가지가 제공되며, 예외를 직접 처리할 때 사용하는 try ~ catch ~ finally 구문, 예외를 이전 호출 위치로 넘기는 throws 구문, 자동으로 close 처리를 해주는 try with resource 문이 있다.

예외가 발생되면 프로그램은 비정상적으로 종료하게 된다. 만약, 프로그램이 종료되지 않고 실행의 흐름을 바꾸고자 한다면 try ~ catch 문으로 직접 예외를 처리해 주면 된다.

try ~ catch 문 맨 마지막에 사용되는 finally 구문은 예외가 발생하든 발생하지 않든 무조건 구동되어야 하는 코드를 작성하는 영역이다.

```
try {
   // 예외 발생 가능성이 있거나, 예외를 반드시 처리해야 하는 코드 작성부
} catch (처리할 예외클래스명 객체 1) {
   // 예외 상황 1에 대한 처리 코드
} catch (처리할 예외클래스명 객체 2) {
   // 예외 상황 2에 대한 처리 코드
} finally {
   // 예외 발생으로 try { } 안의 코드가 중단되더라도 반드시 실행해야 하는 코드
}
```

하나 이상의 catch 문을 사용할 경우에는 반드시 처리할 예외클래스들의 상속관계를 고려하여 최하위 후손 클래스부터 기술해 주어야 한다.

발생한 예외에 대한 정보를 확인하려면 에러 객체를 직접 출력하거나(예외객체.toString( )) 메시지를 출력하거나 (예외객체.getMessage( )) 또는 실행 흐름상의 예외 발생 단계를 추적하여 출력하는 (예외객체.printStackTrace( )) 코드를 추가할 수도 있다.

test.exception.TestException1.java

```
01  package test.exception;
02
03  public class TestException1 {
04     public static void main(String[] args) {
05        int arr[] = new int[3];
06
07        try {
08           for(int i=0; i<arr.length; i++)
09              arr[i] = i;
10
11           for(int i=0; i<=arr.length; i++)
12              System.out.println(arr[i]);
13        } catch(ArrayIndexOutOfBoundsException e) {
14           System.out.println("예외 : 사용불가 인덱스 접근");
15           e.printStackTrace(); // System.out.println(e);
16        } finally {
17           System.out.println("반드시 수행하는 코드");
18        }
     }
  }
```

```
0
1
2
예외 : 사용불가 인덱스 접근
java.lang.ArrayIndexOutOfBoundsException: 3
반드시 수행하는 코드
        at test.Test.main(Test.java:12)
```

# 3 예외 처리 미루기 (throws)

이 방법을 사용하면 해당 메소드 안에서 발생한 예외 처리에 대한 책임을 메소드를 호출한 쪽으로 떠넘길 수 있다. 컴파일 시에는 문제가 없지만 예외 발생 시에 호출하는 코드에서 예외를 처리하지 못 한다면 프로그램은 비정상 종료를 하게 된다. 호출하는 입장에서 선언된 예외를 보고 발생할 수 있는 예외를 미리 파악할 수 있고 또한 반드시 처리해야 하므로 프로그램을 견고하게 한다.

test.exception.TestThrows.java

```
01  package test.exception;
02
03  import java.io.IOException;
04
05  public class TestThrows {
06    public static void main(String[] args) {
07      IOSample test = new IOSample( );
08      try {
09        test.input();
10        test.output();
11      } catch (IOException e) {
12        e.printStackTrace();
13      } catch (ArrayIndexOutOfBoundsException e) {
14        e.printStackTrace();
15      }
16    }
17  }
```

test.exception.IOSample.java

```
01  package test.exception.;
02
03  import java.io.IOException;
04
05  public class IOSample {
06      private int num;
07
08      public void input() throws IOException {
09          num = System.in.read();
10      }
11      public void output() {
```

```
12              System.out.println(num);
13          } }
```

# 4 예외 던지기 (throw)

원하는 조건 상황에서 필요할 경우 예외 객체를 생성시키면서 예외를 발생시킬 수 있다. 자바에서 제공되는 예외 클래스이든 개발자가 만든 사용자 정의 예외 클래스이든 모두 생성하고 발생시킬 수 있다.

test.exception.ThrowExMain.java

```
01  package test.exception;
02
03  import exception.sample.ThrowEx;
04
05  public class ThrowExMain {
06      public static void main(String[] args) throws NullPointerException {
07              ThrowEx ex = new ThrowEx();
08              ex.printStr(5);
09      }
10  }
```

exception.sample.ThrowEx.java

```
01  package exception.sample;
02
03  public class ThrowEx {
04     String str = null;
05
06     public void printStr(int cnt) throws NullPointerException{
07        if( str == null ) {
08           System.out.println("str이 null이다");
09           throw new NullPointerException();
10        }
11        for(int i=0; i<cnt; i++) {
12              System.out.println(str);
13        }
14     }
15  }
```

# 5 사용자 정의 예외클래스 만들기

자바에서 제공해 주는 예외 클래스가 사용하려는 목적에 맞지 않을 때 직접 예외 클래스를 만들어 사용할 수 있다. 사용자 정의 예외 클래스를 만들 때 java.lang.Exception 클래스를 반드시 상속받으면 된다.

exception.sample.EvenException.java

```
01 package exception.sample;
02
03 public class EvenException extends Exception {
04     private Integer number = null;
05
06     public EvenException (int number) {
07        this.number = number;
08     }
09
10     @Override
11     public String getMessage() {
12         if(number == null) {
13             return "숫자가 아닙니다";
14         } else {
15             return Integer.toString(number) + "는 짝수가 아닙니다";
16         }
17     }
18 }
```

exception.sample.CheckEven.java

```
01 package exception.sample;
02
03 public class CheckEven {
04    private int num;
05
06    public void setNum(int num) {
07       this.num = num;
08    }
09
10    public int printEven(int number) throws EvenException {
11       this.num = number;
12
```

```
13        if(num % 2 != 0) {
14           throw new EvenException(num);
15        } else {
16           return num;
17        }
18     }
19  }
```

test.exception.ExMain.java

```
01  package test.exception;
02
03  import exception.sample.*;
04
05  public class ExMain {
06  public static void main(String[] args) {
07        CheckEven ce = new CheckEven();
08
09        int number1 = 100;
10        int number2 = 99;
11
12        try {
13           System.out.println(ce.printEven(number1));
14           System.out.println(ce.printEven(number2));
15        } catch (EvenException e) {
16           e.printStackTrace();
17        }
18     }
19  }
```

```
100
EvenException: 99는 짝수가 아닙니다
        at CheckEven.printEven(CheckEven.java:12)
        at Test.main(Test.java:11)
```

# Chapter 4. 입출력(IO)

## 1 입출력

입출력이란 I/O(Input/Output)라고도 부르며 컴퓨터 내부 또는 외부 장치와 프로그램이 데이터를 주고 받는 것이다

### 1.1 스트림

자바에서 입출력을 하기 위해 두 대상을 연결하고 데이터를 전송할 수 있게 해주는 것

단방향 통신만 가능, 입력과 출력을 동시에 실행하기 위해서는 입력 스트림과 출력 스트림이 별도로 존재해야 한다.

### 1.2 스트림의 특징

① FIFO 구조
먼저 들어간 데이터가 먼저 나오는 형태로 데이터의 순서가 바뀌지 않는다

② 단방향
하나의 스트림에서 입출력이 동시에 이루어 지지 않는다
입력과 출력에 각각 스트림을 열어 사용해야 한다

③ 지연 가능
스트림 내의 데이터가 모두 전송되기 전까지 프로그램이 지연 상태에 빠질 수 있다

### 1.3 스트림의 분류

- **용도에 의한 분류**

① 1차 스트림 : 디바이스에 직접 연결되는 스트림
② 2차 스트림 : 1차 스트림과 연결을 통해 디바이스에 연결되는 스트림

- **전송 방향에 의한 분류**

① 입력 스트림 : 디바이스로부터 데이터를 읽어오는 스트림
② 출력 스트림 : 디바이스로 데이터를 출력하는 스트림

- **전송 단위에 의한 분류**

① 바이트 스트림 : 1 Byte 단위로 입력, 출력하는 스트림
② 문자 스트림 : 한 문자(2 Byte) 단위로 입력, 출력하는 스트림

- **보조 스트림**

스트림의 기능을 향상시키거나 새로운 기능을 추가시킴
직접적인 데이터 입출력을 할 수 없다

## 1.4 File 클래스

파일이나 디렉터리를 추상화한 클래스

가장 많이 사용되는 입출력 대상

### 1.4.1 File 클래스의 생성자와 메소드

| 생성자 | 설명 |
|---|---|
| File(String fileName) | 주어진 파일 이름을 가지는 객체 생성 |
| File(String pathname, String fileName)<br>File(File pathname, String fileName) | 파일의 경로와 이름을 따로 지정하여 객체 생성 |
| File(URI uri) | 지정된 uri로 파일 생성 |

| 메소드 | 설명 |
|---|---|
| String getName() | 파일 이름을 String으로 반환 |
| String getPath() | 파일의 경로를 String으로 반환 |
| String getAbsolutePath()<br>File getAbsoluteFile() | 파일의 절대경로를 String으로 반환<br>파일의 절대경로를 File로 반환 |
| String getParent()<br>File getParentFile() | 파일의 조상 디렉토리를 String으로 반환<br>파일의 조상 디렉토리를 File로 반환 |
| String getCanonicalPath()<br>File getCanonicalFile() | 파일의 정규경로를 String으로 반환<br>파일의 정규경로를 File로 반환 |

# 2 바이트 스트림

바이트단위로 데이터를 전송

바이트단위로 구성된 파일(동영상, 이미지, 음악 등)을 처리하기에 적합한 스트림

바이트 스트림의 최상위 클래스는 InputStream과 OutputStream이다

자손 클래스들은 입출력에 필요한 추상 메소드를 자신에 맞게 구현해 놓았다

## 2.1 입출력 대상에 따른 바이트 스트림

| 입력스트림 | 출력스트림 | 입출력 대상 |
|---|---|---|
| FileInputStream | FileOutputStream | 파일 |
| ByteArrayInputStream | ByteArrayOutputStream | 메모리(byte배열) |
| PipedInputStream | PipedOutputStream | 프로세스(프로세스 간 통신) |

| | | |
|---|---|---|
| AudioInputStream | AudioOutputStream | 오디오장치 |

## 2.2 InputStream과 OutputStream

바이트 스트림의 최상위 클래스로써 둘 다 추상 클래스이다

### 2.2.1 InputStream에 정의된 메소드

| 메소드 | 설명 |
|---|---|
| int available() | 현재 읽을 수 있는 바이트 수를 반환 |
| void close() | 스트림을 닫음 |
| void mark(int readlimit) | 스트림에서 현재 위치를 표시, reset()에 의해 돌아감 |
| boolean markSupported() | mark()와 reset() 지원 여부 확인 |
| abstract int read() | 스트림에서 1바이트를 읽음(0~255 아스키코드 값)<br>더 이상 읽을 데이터가 없으면 -1 반환 |
| int read(byte[] b) | 배열 b의 크기만큼 읽어서 배열을 채우고 읽은 데이터 수를 반환 |
| int read(byte[] b, int off, int len) | 최대 len만큼의 바이트를 읽어서 배열 b의 지정된 위치부터 저장 |
| void reset() | mark() 를 마지막으로 호출한 위치로 이동 |
| long skip(long n) | n 바이트 만큼 데이터를 건너뛰고 바이트 수를 반환 |

### 2.2.2 OutputStream에 정의된 메소드

| 메소드 | 설명 |
|---|---|
| void close() | 스트림을 닫음 |
| void flush() | 스트림 버퍼의 내용을 모두 내보냄 |
| abstract void write(int i) | 주어진 값을 출력 |
| void write(byte b[]) | 배열 b에 저장된 내용을 모두 출력 |
| void write(byte b[], int index, int size) | 배열 b에 저장된 내용을 index부터 size만큼 출력 |

## 2.3 ByteArrayInputStream과 ByteArrayOutputStream

바이트 배열에 데이터를 입출력하는 스트림

다른 디바이스로 입출력하기 전에 임시로 변환하여 저장하는 용도로 사용

스트림의 종류가 다르더라도 사용하는 방법은 비슷

입출력에 사용되는 자원이 메모리 밖에 없으므로 close()를 반드시 하지 않아도 가비지 컬렉터가 자동으로 자원을 회수한다

## 2.4 FileInputStream과 FileOutputStream

바이트 단위로 파일을 통한 입출력 스트림

생성자의 인자로 File 객체를 이용하거나 파일의 이름을 직접 String 형태로 사용
파일이 없을 경우도 있으므로 예외처리를 해야 한다

### 2.4.1 FileInputStream의 생성자

| 생성자 | 설명 |
|---|---|
| FileInputStream(String filePath) | 지정된 파일이름을 가진 실제 파일과 입력 스트림 생성 |
| FileInputStream(File fileObj) | 지정된 파일 객체와 입력 스트림 생성 |
| FileInputStream(FileDescriptor fdObj) | 파일 디스크럽터로 입력 스트림 생성 |

### 2.4.2 FileOutputStream의 생성자

| 생성자 | 설명 |
|---|---|
| FileOutputStream(String filepath) | 지정된 파일이름을 가진 실제 파일과 출력 스트림 생성 |
| FileOutputStream(String filepath, boolean append) | 지정된 파일이름을 가진 실제 파일과 출력 스트림 생성. 추가 모드를 설정 |
| FileOutputStream(File fileObj) | 지정된 파일 객체와 출력 스트림 생성 |
| FileOutputStream(File fileObj, boolean append) | 지정된 파일 객체와 출력 스트림 생성. 추가 모드를 설정 |
| FileOutputStream(FileDescriptor fdObj) | 파일 디스크럽터로 출력 스트림 생성 |

[예제_FileStream을 이용한 파일 복사]

# 3 바이트 보조 스트림

## 3.1 보조 스트림의 종류

| 입력스트림 | 출력스트림 | 설명 |
|---|---|---|
| FilterInputStream | FilterOutputStream | 입출력 스트림 보조 |
| BufferedInputStream | BufferedOutputStream | 버퍼를 이용한 성능 향상 |
| DataInputStream | DataOutputStream | 기본형 단위 데이터 처리 |
| SequenceInputStream | 없음 | 여러 개의 스트림을 하나로 연결 |
| ObjectInputStream | ObjectOutputStream | 객체 단위로 입출력<br>객체 직렬화와 관련 |
| 없음 | PrintStream | 버퍼를 이용하여 추가적인 print관련 기능 |
| PushbackInputStream | 없음 | 버퍼를 이용해서 읽어 온 데이터를 다시 되돌리는 기능 |

## 3.2 FilterInputStream과 FilterOutputStream

InputStream / OutputStream의 자손

모든 보조 스트림의 부모 클래스

### 3.2.1 FilterInputStream과 FilterOutputStream 의 생성자

FilterInputStream 생성자의 접근제어자가 protected이기 때문에 직접 객체를 생성해 사용할 수 없고 상속을 통해 오버라이딩되어야 한다

| 생성자 | 설명 |
|---|---|
| protected FilterInputStream(InputStream in) | InputStream의 객체를 이용해 특정 기능을 수행하는 FilterInputStream 객체를 생성 |
| public FilterOutputStream(OutputStream out) | OutputStream 의 객체를 이용해 특정 기능을 수행하는 FilterOutputStream 객체를 생성 |

| 입력스트림 | 출력스트림 | 입출력 대상 |
|---|---|---|
| FileInputStream | FileOutputStream | 파일 |
| ByteArrayInputStream | ByteArrayOutputStream | 메모리(byte배열) |
| PipedInputStream | PipedOutputStream | 프로세스(프로세스 간 통신) |
| AudioInputStream | AudioOutputStream | 오디오장치 |

## 3.3 BufferedInputStream과 BufferedOutputStream

입출력 효율을 높이기 위해 버퍼를 사용하는 보조스트림

한 바이트씩 입출력하는 것 보다 버퍼를 이용해서 한번에 여러 바이트를 입력하는 것이 효율적이고 빠르기 때문에 대부분의 입출력 작업에 사용

- BufferedInputStream

  원하는 자료를 1바이트 단위로 읽는 read() 메소드를 수행하면 내부적으로 버퍼를 준비하고 버퍼의 크기만큼 한꺼번에 데이터를 읽고 버퍼로부터 1바이트씩 읽어들여 프로그램으로 전달

- BufferedOutputStream

  1바이트씩 출력하면 시스템 버퍼에 출력을 쌓아 두고 버퍼가 모두 채워지거나 flush(), close() 명령을 사용하면 버퍼의 모든 내용을 출력

### 3.3.1 BufferedInputStream 생성자

| 생성자 | 설명 |
|---|---|
| BufferedInputStream(InputStream in) | InputStream에 대한 BufferedInputStream 객체 생성 |
| BufferedInputStream(InputStream in, int size) | InputStream에 대한 BufferedInputStream 객체 생성. 내부 버퍼의 크기를 Size 값으로 설정 |

### 3.3.2 BufferedOutputStream 생성자

| 생성자 | 설명 |
|---|---|
| BufferedOutputStream(OutputStream out) | OutputStream에 대한 BufferedOutputStream 객체 생성 |
| BufferedOutputStream(OutputSteam out, int size) | OutputStream에 대한 BufferedOutputStream 객체생성. 내부 버퍼의 크기를 Size 값으로 설정 |

FIleStream 같은 경우 1차 스트림이기 때문에 미리 만들어져 있는 스트림 없이 자신의 객체를 생성해 입출력을 시도할 수 있지만 BufferedStream 객체는 2차 스트림이기 때문에 미리 만들어진 스트림을 생성자의 인자로 받아 객체를 생성한다

## 3.4 DataInputStream과 DataOutputStream

자바의 기본자료형 데이터를 입출력하는 기능을 제공하는 스트림

### 3.4.1 DataInputStream과 DataOutputStream 생성자

| 생성자 | 설명 |
|---|---|
| DataInputStream(InputStream in) | InputStream을 인자로 DataInputStream을 생성 |
| DataOutputStream(OutputStream out) | OutputStream을 인자로 DataOutputStream을 생성 |

### 3.4.2 DataInputStream 메소드

| 메소드 | 설명 |
|---|---|
| boolean readBoolean() | 스트림에서 읽은 boolean을 반환 |
| byte readByte() | 스트림에서 읽은 byte를 반환 |
| char readChar() | 스트림에서 읽은 char를 반환 |
| double readDouble | 스트림에서 읽은 double을 반환 |
| float readFloat() | 스트림에서 읽은 float을 반환 |
| long readLong() | 스트림에서 읽은 long을 반환 |
| short readShort() | 스트림에서 읽은 short를 반환 |
| int readInt( | 스트림에서 읽은 int를 반환 |
| void readFully(byte[] buf) | 스트림에서 buf 크기만큼 바이트를 읽어 buf 저장 |
| void readFully(byte[] buf, int off, int len) | 스트림에서 len 길이만큼 바이트를 읽어 buf의 off 위치에 저장 |
| String readUTF() | UTF-8 형식으로 쓰여진 문자를 읽어 문자열로 반환 |
| static String readUTF(DataInput in) | 입력스트림 in에서 UTF-8 형식으로 쓰여진 문자를 읽어 문자열로 반환 |
| int skipBytes(int n) | 현재 읽고 있는 위치에서 n 만큼 건너뜀 |

### 3.4.3 DataOutputStream 메소드

| 메소드 | 설명 |
|---|---|
| void flush() | 버퍼를 출력하고 비움 |
| int size() | 스트림에 출력된 바이트 크기를 반환 |
| void writeBoolean(boolean b) | boolean을 1바이트 값으로 출력 |
| void writeByte(int i) | int를 4바이트 값으로 상위 바이트 먼저 출력 |
| void writeBytes(String s) | 문자열을 바이트 순으로 출력 |
| void writeChar(int i) | char를 2바이트 값으로 상위 바이트 먼저 출력 |
| void writeChars(String s) | String 문자열을 char형으로 출력 |
| void writeDouble(double d) | Double 클래스의 doubleToBits()를 사용하여 long으로 변환하여 long 값을 8바이트수량으로 상위바이트 먼저 출력 |
| void writeFloat(float f) | Float 클래스의 floatToBits()를 사용하여 변환하여 int 값을 4바이트 수량으로 상위 바이트 먼저 출력 |
| void writeInt(int i) | int의 상위 바이트 먼저 출력 |
| void writeLong(long l) | long 상위 바이트 먼저 출력 |
| void writeShort(shrot s) | short형의 인자값 출력 |
| void writeUTF(String s) | UTF-8 형식으로 문자열 출력 |

## 3.5 SequenceInputStream

여러 개의 입력 스트림을 연속적으로 연결해서 하나의 스트림으로 처리

생성자를 제외하고 다른 입력 스트림과 기능이 동일하다

큰 파일을 여러 개의 작은 파일로 나누었다가 하나의 파일로 합치는 작업 등을 할 때 유용하다

### 3.5.1 SequenceInputStream 생성자

| 생성자 | 설명 |
|---|---|
| SequenceInputStream(Enum e) | 열거체 e에 저장된 순서대로 입력 스트림 연결 |
| SequenceInputStream(InputStream in1, InputStream in2) | 두 개의 입력 스트림을 하나로 연결 |

# 4 문자 스트림

문자단위로 데이터 전송
바이트 스트림과 사용방법은 거의 같다
영어 이외의 문자에 대한 처리와 인코딩을 내부에서 처리
문자 스트림의 최상위 클래스는 Reader와 Writer이다

## 4.1 문자 스트림과 바이트 스트림 비교

InputStream이 Reader로 OutputStream이 Writer에 대응
이름만 조금 다를 뿐 활용방법은 거의 같다

- 스트림 비교

| 바이트 스트림 | 문자 스트림 |
|---|---|
| FileInputStream<br>FileOutputStream | FileReader<br>FileWriter |
| ByteArrayInputStream<br>ByteArrayOutputStream | CharArrayReader<br>CharArrayWriter |
| PipedInputStream<br>PipedOutputStream | PipedReader<br>PipedWriter |
| StringBufferInputStream<br>StreainBufferOutputStream | StringReader<br>StringWriter |

* StringBufferInputStream과 StringBufferOutputStream은 JDK 1.1부터 사용되지 않음

- InputStream과 Reader 메소드 비교

| InputStream | Reader |
|---|---|
| abstract int read() | int read() |
| int read(byte[] b) | int read(char[] cbuf) |
| int read(byte[] b, int off, int len) | abstract int read(char[] cbuf, int off, int len) |

- OutputStream과 Writer 메소드 비교

| OutputStream | Writer |
|---|---|
| abstract void write(int b) | void write(int c) |
| void write(byte[] b) | void write(char[] cbuf) |
| void write(byte[] b, int off, int len) | abstract void write(char[] cbuf, int off, int len) |
| - | void write(String str) |
| - | void write(String str, int off, int len) |

- 보조 스트림 비교

| 바이트 보조 스트림 | 문자 보조 스트림 |
|---|---|
| BufferedInputStream<br>BufferedOutputStream | BufferedReader<br>BufferedWriter |
| FilterInputStream<br>FilterOutputStream | FilterReader<br>FilterWriter |
| LineNumberInputStream | LineNumberReader |
| PrintStream | PrintWriter |
| PushbackInputStream | PushbackReader |

* LineNumberInputStream은 JDK 1.1부터 사용되지 않음

## 4.2 Reader와 Writer

문자 스트림의 최상위 클래스로써 둘 다 추상 클래스이다

### 4.2.1 Reader에 정의된 메소드

| 메소드 | 설명 |
|---|---|
| abstract void close() | 스트림을 닫음 |
| void mark(int limit) | 스트림의 현재 위치를 표시 |
| boolean markSupported() | mark()와 reset() 지원 여부 확인 |
| int read() | 스트림에서 단일 문자를 읽음 |
| int read(char buf[]) | 스트림에서 buf[] 크기 만큼 읽어 buf에 저장하고 읽은 문자 수를 반환 |
| abstract int read(char buf[], int off, int len) | 스트림에서 len 만큼을 읽어 buf[]의 off 위치에 저장하고 읽은 문자 수를 반환 |
| int read(CharBuffer target) | CharBuffer 형인 target에 문자열을 읽음 |
| boolean ready() | 스트림이 준비되었는지 반환 |
| void reset() | 스트림을 mark된 위치로 되돌림 |
| long skip(long l) | 주어진 개수 l 만큼의 문자를 건너뜀 |

### 4.2.2 Writer에 정의된 메소드

| 메소드 | 설명 |
|---|---|
| Writer append(char c) | 스트림에 c 추가 |
| Writer append(CharSequence csq) | 스트림에 CharSequence csq를 추가 |
| Writer append(CharSequence csq, int start, int | 스트림에 CharSequence csq의 start부터 end까지의 문자를 추가 |

| | |
|---|---|
| end) | |
| abstract void close() | 스트림을 닫음 |
| abstract void flush() | 스트림 버퍼에 남은 데이터를 출력 |
| void write(String s) | 주어진 문자열 s를 출력 |
| void write(char buf[]) | buf의 내용을 출력 |
| void write(char buf[], int off, int len) | buf의 off위치부터 len만큼의 문자를 출력 |
| void write(String s, int off, int len) | 문자열 s에 있는 문자들을 off위치부터 len만큼 출력 |

## 4.3 FileReader와 FileWriter

파일에 저장된 바이트를 유니코드 문자로 변환해서 입력

출력할 유니코드 문자를 디폴트 문자 인코딩의 바이트로 변환해서 파일에 출력

### 4.3.1 FileReader 생성자

| 생성자 | 설명 |
|---|---|
| FileReader(File file) | 지정된 파일에 대한 입력 스트림 생성 |
| FileReader(FileDescriptor fd) | 지정된 FileDescriptor에 대한 입력 스트립을 생성 |
| FileReader(String fileName) | 지정된 파일이름을 가진 파일에 대한 입력 스트림을 생성 |

### 4.3.2 FileWriter 생성자

| 생성자 | 설명 |
|---|---|
| FileWriter(File file) | 파일로 지정된 파일에 대한 출력 스트림을 생성 |
| FileWriter(File file, boolean append) | 지정된 파일에 추가 모드를 설정하여 출력 스트림을 생성 |
| FileWriter(FileDescriptor fd) | 지정된 FileDescriptor에 대한 출력 스트립을 생성 |
| FileWriter(String fileName) | 지정된 파일이름을 가진 파일에 대한 출력 스트림을 생성 |
| Writer(String fileName, boolean append) | 지정된 파일이름을 가진 파일에 추가 모드를 설정하여 출력 스트림을 생성 |

# 5 문자 보조 스트림

## 5.1 BufferedReader와 BufferedWriter

입출력의 효율을 높이기 위해 버퍼를 이용한 보조 스트림

### 5.1.1 BufferedReader와 BufferedWriter의 생성자

| 생성자 | 설명 |
|---|---|
| BufferedReader(Reader in) | 문자 입력 스트림 in에 대해 기본 크기의 버퍼를 갖는 객체를 생성 |
| BufferedReader(Reader in , int size) | 문자 입력 스트림 in에 대해 size 크기의 버퍼를 갖는 객체를 생성 |
| BufferedWriter(Writer out) | 문자 출력 스트림 out에 대해 기본 크기의 버퍼를 갖는 객체를 생성 |
| BufferedWriter(Writer out, int size) | 문자 출력 스트림 out에 대해 size 크기의 버퍼를 갖는 객체를 생성 |

### 5.1.2 BufferedReader 의 메소드

| 메소드 | 설명 |
|---|---|
| String readLine() | 한 줄을 읽는다. "₩n", "₩r"을 만날 때 까지 읽음 |

한 줄씩 읽어 처리하므로 효율적인 문자 읽기가 가능하다

### 5.1.3 BufferedWriter의 메소드

| 메소드 | 설명 |
|---|---|
| String newLine() | 줄을 바꿈 |

문자를 출력할 때 줄 바꿈이 필요할 때 사용할 수 있다

## 5.2 InputStreamReader와 OnputStreamWriter

바이트 스트림과 문자 스트림을 연결할 수 있도록 도와주는 스트림

인코딩 방식은 특정할 수도 있고 플랫폼의 기본 인코딩을 사용하기도 한다

### 5.2.1 InputStreamReader의 생성자

| 생성자 | 설명 |
|---|---|
| InputStreamReader(InputStream in) | 기본 인코딩을 사용하는 객체를 생성 |
| InputStreamReader(InputStream in, String charsetName) | 지정된 인코딩을 사용하는 객체를 생성 |

### 5.2.2 OutputStreamWriter의 생성자

| 생성자 | 설명 |
|---|---|
| OutputStreamWriter(OutputStream out) | 기본 인코딩을 사용하는 객체를 생성 |
| OutputStreamWriter(OutputStream out, String | 지정된 인코딩을 사용하는 객체를 생성 |

| | |
|---|---|
| charsetName) | |

### 5.2.3 InputStreamReader 의 메소드

| 메소드 | 설명 |
|---|---|
| void close() | 스트림을 닫는다 |
| String getEncoding() | 인코딩의 표준 이름을 얻는다. |
| int read() | 문자 하나를 읽는다. (없을 경우 -1 반환) |
| int read(char[] cbuf, int offset, int length) | cbuf의 버퍼에 offset부터 length길이만큼의 문자 |
| boolean ready() | InputStream에서 문자가 있는지 읽을 수 있는지 여부를 확인 |

### 5.2.4 OutputStreamWriter의 메소드

| 메소드 | 설명 |
|---|---|
| void close() | 스트림을 닫는다. |
| void flush() | 스트림 버퍼를 비우고 출력 |
| String getEncoding() | 인코딩의 표준 이름을 얻는다 |
| void write(char[] cbuf, int off, int len) | cbuf의 버퍼에 off부터 len만큼을 담아 문자를 출력 |
| void write(int c) | c개의 문자를 출력 |
| void write(String str, int off, int len) | 문자열 str의 off부터 len만큼을 출력 |

# 6 직렬화

데이터를 파일에 쓰거나, 네트워크를 타고 다른 곳에 전송할 때는 데이터를 바이트 단위로 분해하여 순차적으로 보내야 하는데 이것을 직렬화(Serialization)라고 한다. 반대의 경우를 역직렬화(Deserialization)라고 한다.

자바에서는 객체를 데이터 스트림으로 만드는 것을 뜻하며, 객체의 데이터를 출력하기 위해 연속적인 데이터로 변환하는 것이다. 스트림으로부터 데이터를 입력받아 객체를 만드는 것을 역직렬화라고 한다.

## 6.1 직렬화가 가능한 객체의 조건

① 기본형 타입
② Serializable 인터페이스를 구현한 클래스
③ 객체 멤버 중 Serializable인터페이스를 구현하지 않은 객체가 존재하면 안 된다
④ transient가 사용된 멤버는 전송되지 않는다

## 6.2 ObjectInputStream과 ObjectOutputStream

직렬화에 ObjectOutputStream을 사용하고, 역직렬화에 ObjectInputStream을 사용
둘 모두 각각 InputStream과 OutputStream을 직접 상속받지만 보조 스트림이다

### 6.2.1 ObjectInputStream과 ObjectOutputStream의 생성자

| 생성자 | 설명 |
|---|---|
| ObjectInputStream(InputStream in) | 입력 스트림으로부터 ObjectInputStream 객체를 생성 |
| ObjectOutputStream(OutputStream out) | 출력 스트림으로부터 ObjectOutputStream 객체를 생성 |

### 6.2.2 ObjectInputStream의 메소드

| 메소드 | 설명 |
|---|---|
| void defaultReadObject() | 스트림에서 static, transient가 아닌 객체를 읽는다 |
| int read() | 데이터를 바이트 단위로 읽는다 |
| int read(byte[] buf, int off, int len) | buf 바이트 배열에 off부터 len까지 읽는다 |
| boolean readBoolean() | 객체의 boolean 값을 읽는다 |
| byte readByte() | 객체의 1 byte를 읽는다 |
| char readChar() | 객체의 1 Char를 읽는다 |
| double readDouble() | 객체에서 1 double을 읽는다 |
| float readFloat() | 객체에서 1 float을 읽는다 |

| | |
|---|---|
| int readInt() | 객체에서 1 int를 읽는다 |
| Long readLong() | 객체에서 1 Long을 읽는다 |
| Short readShort() | 객체에서 1 Short를 읽는다 |
| Object readObject() | 객체에서 Object를 읽는다 |
| String readUTF() | String을 UTF-8 방식으로 읽는다 |

* 역직렬화를 직접 구현할 때 주로 사용하며, defaultReadObject()는 자동 역직렬화 수행

### 6.2.3 InputStreamReader 의 메소드

| ObjectOutputStream 메소드 | 설명 |
|---|---|
| defaultWriteObject() | 스트림에 static, transient가 아닌 객체를 출력 |
| write(byte[] buf) | buf를 출력 |
| write(byte[] buf, int off, int len) | buf의 off부터 len 길이만큼을 스트림에 출력 |
| write(int val) | val 바이트만큼 스트림에 출력 |
| writeBoolean(boolean val) | val을 스트림에 출력 |
| writeByte(int val) | val을 byte로 스트림에 출력 |
| writeBytes(String str) | str을 sequence 바이트로 스트림에 출력 |
| writeChar(int val) | val을 Char로 스트림에 출력 |
| writeChars(String str) | str을 sequence char로 스트림에 출력 |
| writeDouble(double val) | val을 Double로 스트림에 출력 |
| writeFloat(float val) | val을 Float으로 스트림에 출력 |
| writeInt(int val) | val을 Int로 스트림에 출력 |
| writeLong(long val) | val을 long로 스트림에 출력 |
| writeObject(Object obj) | Obj 객체를 스트림에 출력 |
| writeShort(int val) | val을 Short로 스트림에 출력 |
| writeUnshared(Object obj) | "unshared" 객체를 스트림에 출력 |
| writeUTF(String str) | 객체의 문자의 인코딩을 UTF-8로 설정 |

* 직렬화를 직접 구현할 때 주로 사용하며, defaultWriteObject()는 자동 직렬화 수행

## 6.3 직렬화 가능한 클래스 만들기

직렬화 하고자 하는 클래스에 java.io.Serializable인터페이스를 구현

Serializable 인터페이스는 빈 인터페이스이므로 상속받은 클래스에서는 따로 구현할 필요가 없다

직렬화 가능한 클래스 멤버 중 보안상 데이터를 직렬화하면 안 되는 경우 transient 키워드를 앞에 붙여 데이터가 null로 전송되도록 할 수 있다

## 6.4 serialVersionUID

Serializable 인터페이스를 implements하면 serialVersionUID가 선언되지 않았다는 Warning이 발생

선언하지 않아도 JVM이 자동 생성하므로 동작은 하지만 직접 선언하는 것을 권장하고 있어 Warning이 발생한다

serialVersionUID는 private static final로 선언

정수값이면 어떤값이든 상관없지만 중복방지를 위해 자바가 설치된 폴더 내의 bin폴더에 포함된 serialver.exe를 이용해 자동 생성된 값을 사용한다

# Chapter 5. 공공데이터 자바에서 활용하기

## 1. 공공데이터 제공 사이트 접속하기

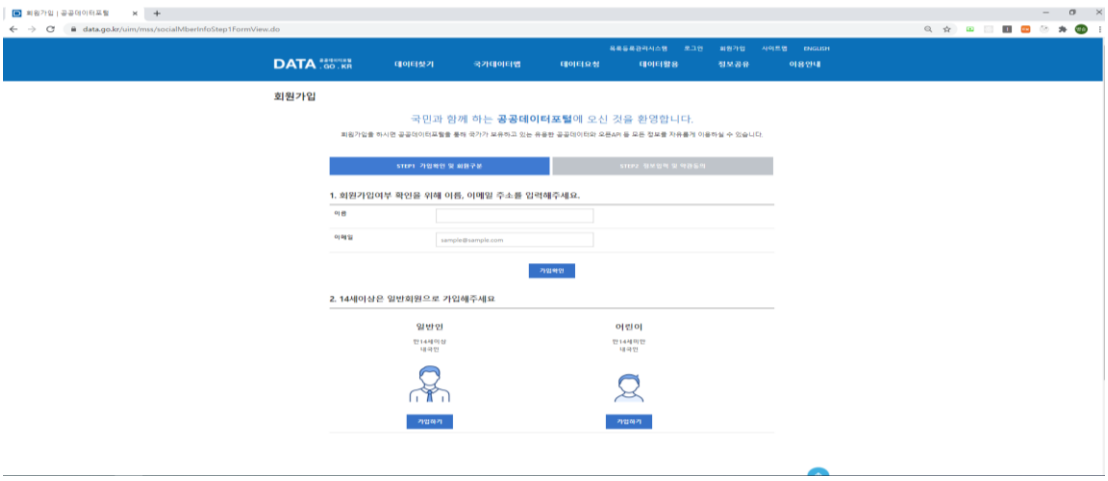
정부에서 제공하는 공공데이터 포털사이트를 이용하면 공공데이터를 손쉬게 이용할 수 있다. 브라우저를 열고 https://www.data.go.kr/index.do 접속하여 정부에서 제공하는 공공데이터를 활용할 수 있다.




중간에 있는 검색창을 이용해서 원하는 데이터를 검색할 수가 있고 세부검색 조건을 아래 있는 상세 조건으로 검색할 수 있다.

## 2. 공공데이터 회원가입하기

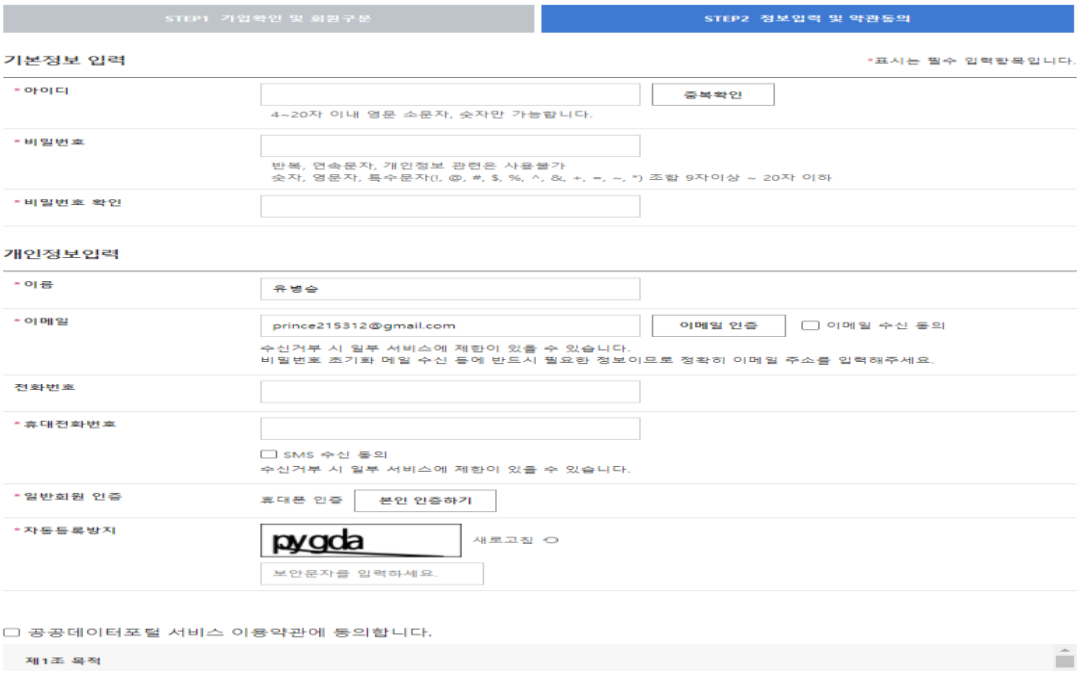
일단 공공데이터포털에서 제공하는 공공데이터를 이용하려면 회원가입을 해야한다.



1. 이름과 사용이 가능한 email 을 입력하고 가입확인을 한다. -> 가입정보가 없음 출력

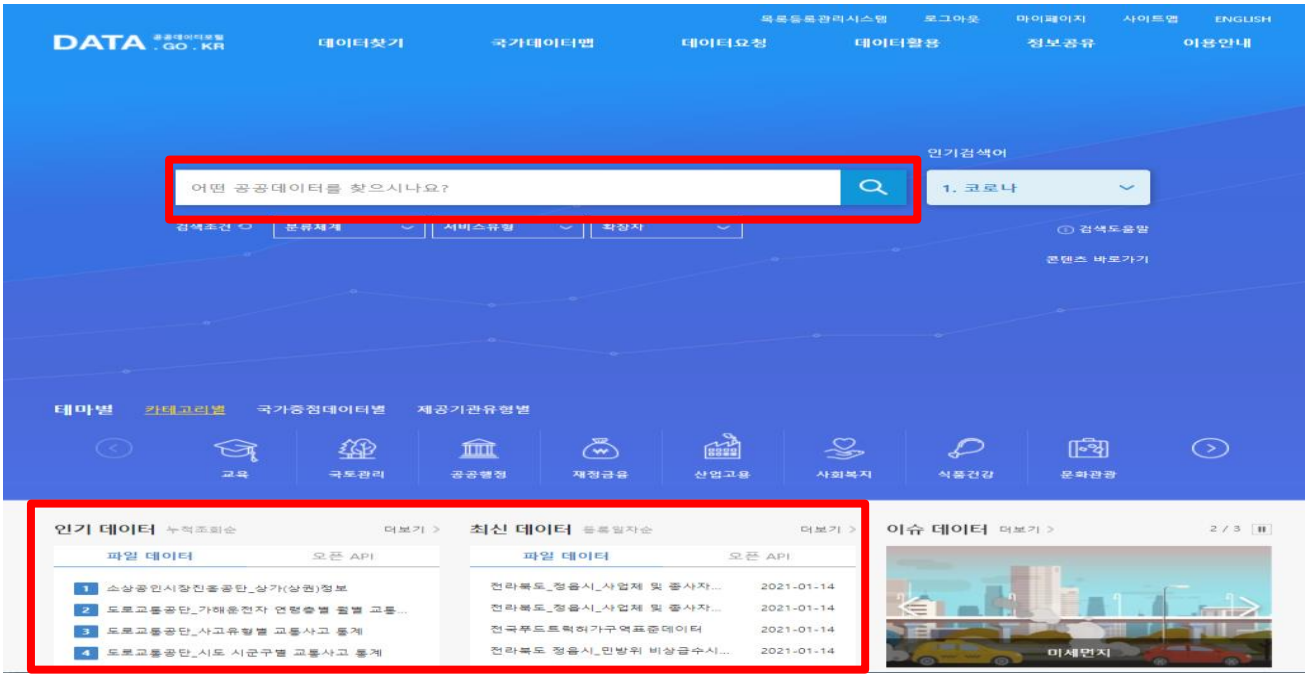
2. 연력에 맞춰 가입하기를 선택한다.

3. 각 항목에 상세 정보를 입력하여 회원가입을 완료한다.



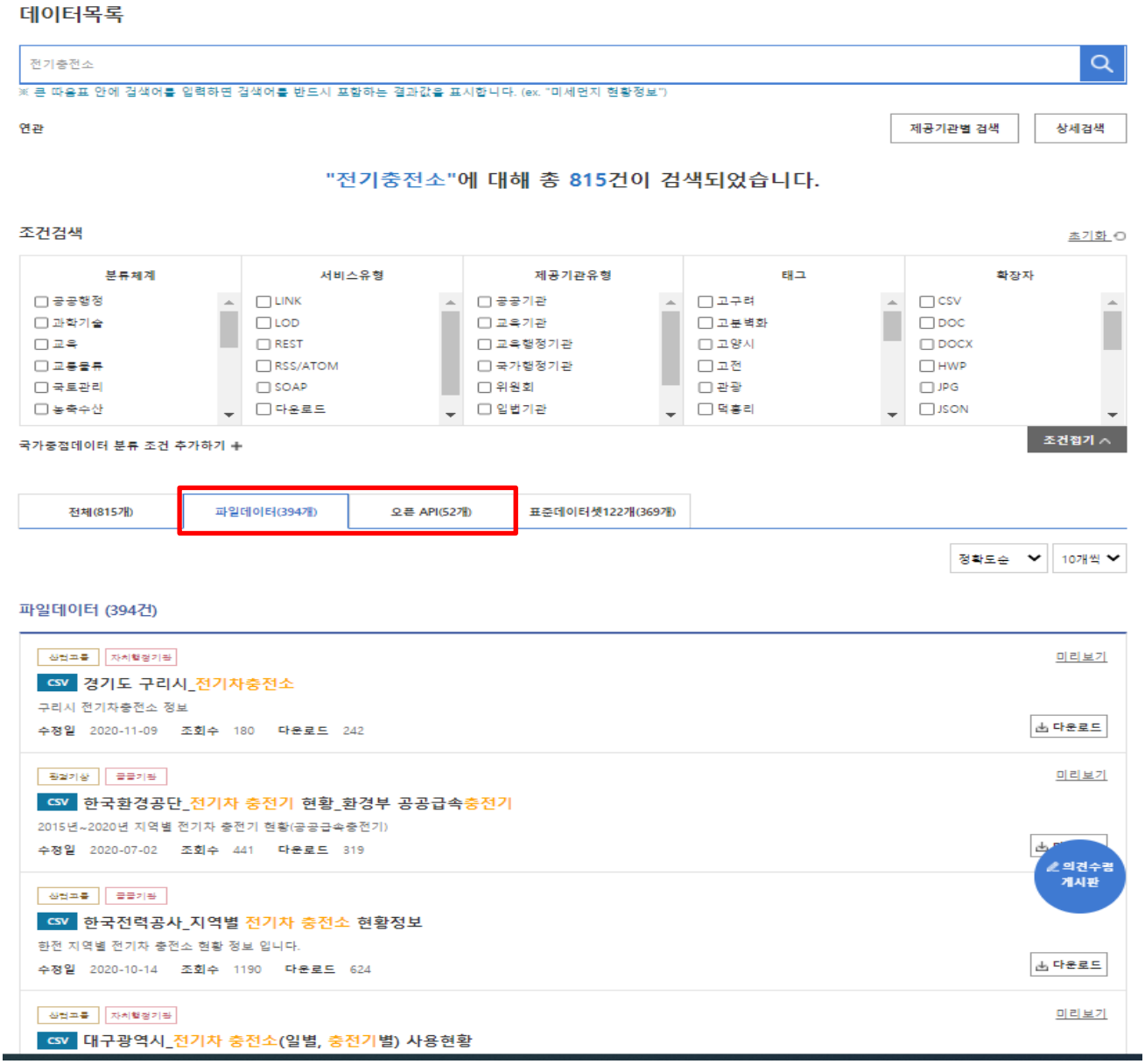

# 3. 공공데이터 조회하기

필요한 데이터는 공공데이터 포털 사이트의 메인화면 조회창을 이용해서 검색할수도 있고 인기데이터, 최신데이터 등 링크로 걸려있는 추천 데이터를 확인할 수 있다.



## 3.1 조회창 이용해서 데이터 검색하기

원하는 데이터 내용을 조회창에 검색을 하면 검색 결과가 나오고 검색결과는 파일데이터, 오픈 API 로 구분하여 확인할 수 있다. 파일데이터는 데이터가 파일로 저장되어있는 형태를 말하는것이고 오픈 API 는 요청주소로 HttpRequest 를 보내서 받아오는 방식을 의미한다. 검색 조건을 상세화하여 필터링도 가능하다.




## 3.2 파일데이터 상세보기

파일데이터 상세보기를 하면 데이터가 파일을 다운로드할 수 있는 버튼이 있고, 데이터 관련 상세내용을 확인할 수 있다. 미리보기를 보면 데이터의 구조와 내용을 미리 확인할 수 있다. 파일로 제공되는 공공데이터는 별로 인증키 없이 다운로드가 가능하다

## 파일데이터 상세

URL 복사

CSV 경기도 구리시_전기차충전소

구리시 전기차충전소 정보

0 0 관심

다운로드

오류신고 및 담당자 문의

### 파일데이터 정보

메타데이터 다운로드

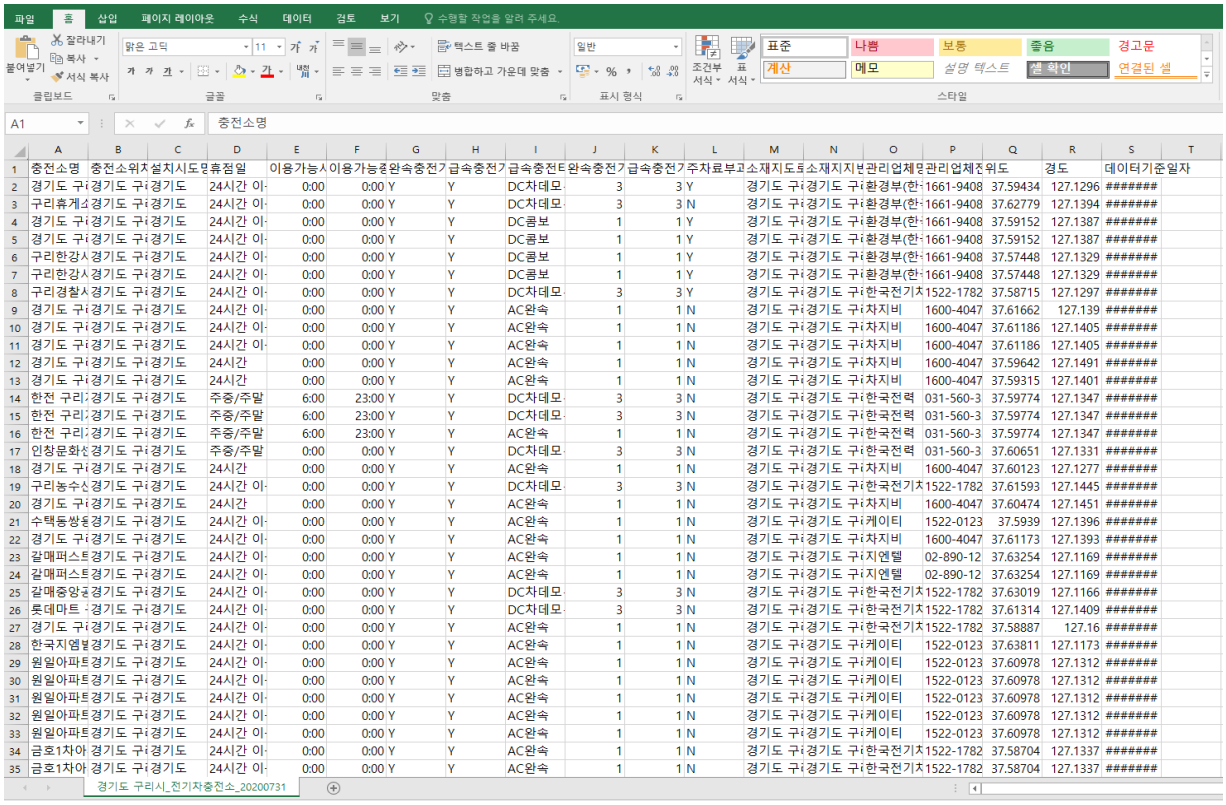
| | | | |
|---|---|---|---|
| 파일데이터명 | 경기도 구리시_전기차충전소_20200731 | | |
| 분류체계 | 산업·통상·중소기업 - 에너지및자원개발 | 제공기관 | 경기도 구리시 |
| 관리부서명 | 정보통신과 | 관리부서 전화번호 | 031-550-2087 |
| 보유근거 | | 수집방법 | |
| 업데이트 주기 | 연간 | 차기 등록 예정일 | 2021-07-31 |
| 매체유형 | 텍스트 | 전체 행 | 44 |
| 확장자 | CSV | 다운로드(바로가기) | 244 |
| 데이터 한계 | | 키워드 | 전기차,전기차충전,구리시 |
| 등록 | 2020-11-09 | 수정 | 2020-11-09 |
| 제공형태 | 공공데이터포털에서 다운로드(원문파일등록) | | |
| 설명 | 구리시 전기차충전소 정보 | | |
| 기타 유의사항 | | | |
| 비용부과유무 | 무료 | 비용부과기준 및 단위 | 건 |
| 이용허락범위 | 이용허락범위 제한 없음 | | |

### 미리보기

※ 파일 데이터의 일부 내용을 제공하고 있으며, 전체 내용이 필요한 경우 해당 파일을 다운로드 받으시기 바랍니다.

| 충전소명 | 충전소위치상세 | 설치시도명 | 휴점일 | 이용가능시작시각 | 이용가능종료시각 | 완속충전가능여부 | 급속충전가능여부 | 급속충전타입구분 | 완속충전기대수 |
|---|---|---|---|---|---|---|---|---|---|
| 경기도 구리시청 | 경기도 구리시 아차산로 439 본관 흡연실 옆 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC차데모+AC3상+DC콤보 | 3 |
| 구리휴게소(외측방향) | 경기도 구리시 서울외곽순환고속도로 32 (사노동) | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC차데모+AC3상+DC콤보 | 3 |
| 경기도 구리시 행정복지센터 | 경기도 구리시 체육관로 74 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC콤보 | 1 |
| 경기도 구리시 행정복지센터 | 경기도 구리시 체육관로 74 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC콤보 | 1 |
| 구리한강시민공원(주차장) | 경기도 구리시 코스모스길14번길 249 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC콤보 | 1 |
| 구리한강시민공원(주차장) | 경기도 구리시 코스모스길14번길 249 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC콤보 | 1 |
| 구리경찰서 | 경기도 구리시 아차산로 359 | 경기도 | 24시간 이용가능 | 00:00 | 00:00 | Y | Y | DC차데모+AC3상+DC콤보 | 3 |

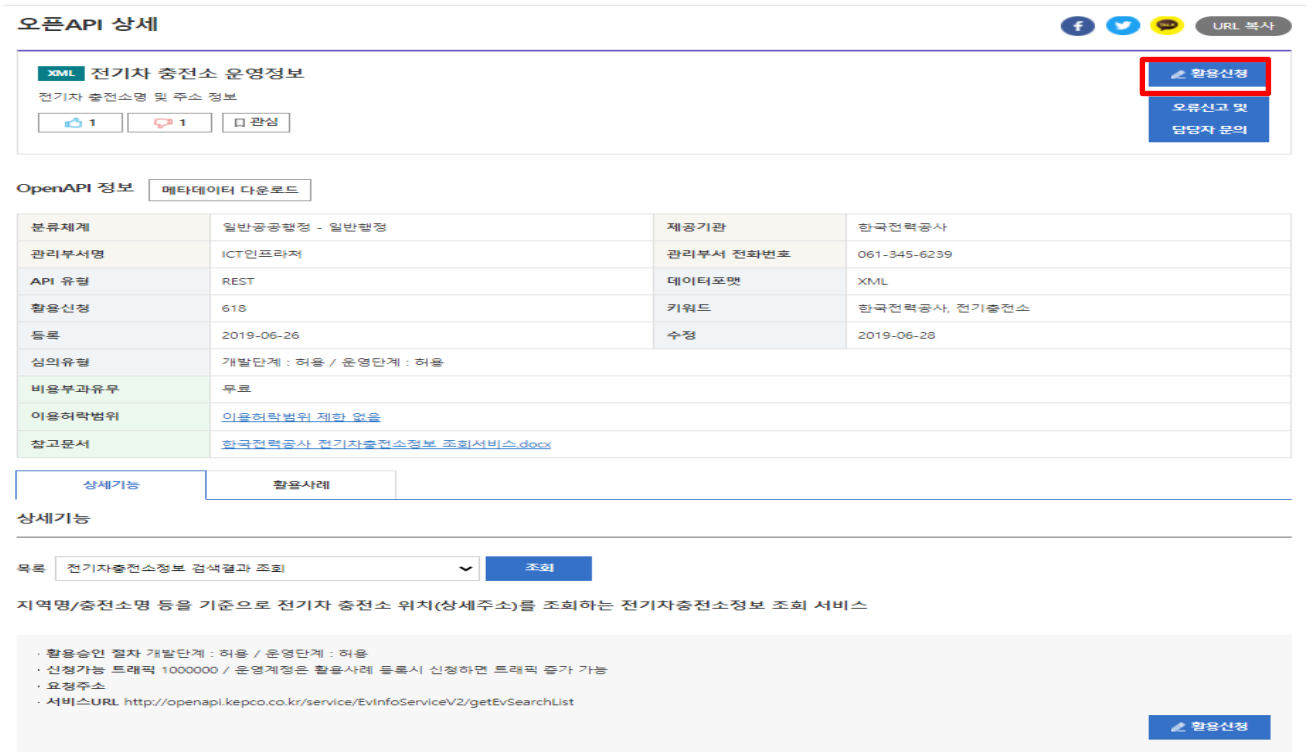
미리보기에 있는 데이터구조와 내용을 보고 데이터 저장방식 및 class 를 설계하면 된다. csv 파일을 다운로드 받으면 엑셀로 내부 데이터 전부를 확인할 수 있음.



| 충전소명 | 충전소위치 | 설치시도명 | 휴점일 | 이용가능시 | 이용가능종 | 완속충전기 | 급속충전기 | 급속충전타 | 완속충전기 | 급속충전기 | 주차료부과 | 소재지도로 | 소재지지번 | 관리업체명 | 관리업체전 | 위도 | 경도 | 데이터기준일자 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | Y | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.59434 | 127.1296 | ######## |
| 구리휴게소 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.62779 | 127.1394 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC콤보 | 1 | 1 | Y | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.59152 | 127.1387 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC콤보 | 1 | 1 | Y | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.59152 | 127.1387 | ######## |
| 구리한강시 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC콤보 | 1 | 1 | Y | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.57448 | 127.1329 | ######## |
| 구리한강시 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC콤보 | 1 | 1 | Y | 경기도 구리 | 경기도 구리 | 환경부(한 | 1661-9408 | 37.57448 | 127.1329 | ######## |
| 구리경찰서 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | Y | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.58715 | 127.1297 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.61662 | 127.139 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.61186 | 127.1405 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.61186 | 127.1405 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.59642 | 127.1491 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.59315 | 127.1401 | ######## |
| 한전 구리지 | 경기도 구리 | 경기도 | 주중/주말 | 6:00 | 23:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전력 | 031-560-3 | 37.59774 | 127.1347 | ######## |
| 한전 구리지 | 경기도 구리 | 경기도 | 주중/주말 | 6:00 | 23:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전력 | 031-560-3 | 37.59774 | 127.1347 | ######## |
| 한전 구리지 | 경기도 구리 | 경기도 | 주중/주말 | 6:00 | 23:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 한국전력 | 031-560-3 | 37.59774 | 127.1347 | ######## |
| 인창문화센 | 경기도 구리 | 경기도 | 주중/주말 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전력 | 031-560-3 | 37.60651 | 127.1331 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.60123 | 127.1277 | ######## |
| 구리농수산 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.61593 | 127.1445 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.60474 | 127.1451 | ######## |
| 수택동쌍용 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.5939 | 127.1396 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 차지비 | 1600-4047 | 37.61173 | 127.1393 | ######## |
| 갈매퍼스트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 지엔텔 | 02-890-12 | 37.63254 | 127.1169 | ######## |
| 갈매퍼스트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 지엔텔 | 02-890-12 | 37.63254 | 127.1169 | ######## |
| 갈매중앙공 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.63019 | 127.1166 | ######## |
| 롯데마트 구 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | DC차데모 | 3 | 3 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.61314 | 127.1409 | ######## |
| 경기도 구리 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.58887 | 127.16 | ######## |
| 한국지엠별 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.63811 | 127.1173 | ######## |
| 원일아파트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.60978 | 127.1312 | ######## |
| 원일아파트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.60978 | 127.1312 | ######## |
| 원일아파트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.60978 | 127.1312 | ######## |
| 원일아파트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.60978 | 127.1312 | ######## |
| 원일아파트 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 케이티 | 1522-0123 | 37.60978 | 127.1312 | ######## |
| 금호1차아 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.58704 | 127.1337 | ######## |
| 금호1차아 | 경기도 구리 | 경기도 | 24시간 이 | 0:00 | 0:00 | Y | Y | AC완속 | 1 | 1 | N | 경기도 구리 | 경기도 구리 | 한국전기차 | 1522-1782 | 37.58704 | 127.1337 | ######## |

## 3.3 오픈 API 상세보기

오픈 API 이용하는 요청 URL 주소와 요청시 파라미터정보, 응답하는 데이터에 대한 정보를 상세하게 보여준다. 오픈 API 활용에 대한 문서가 있어 이를 참고하면 손쉽게 이용할 수 있다. 그리고 오픈 API 를 통해서 공공데이터를 이용하려면 활용신청을 해서 인증키를 받아서 요청을 보내야한다.



요청변수(Request Parameter)

| 항목명(국문) | 항목명(영문) | 항목크기 | 항목구분 | 샘플데이터 | 항목설명 |
|---|---|---|---|---|---|
| 서비스키 | ServiceKey | 400 | 필수 | 공공데이터포털에서 받은 인증키 | 공공데이터포털에서 받은 인증키 |
| 페이지 번호 | pageNo | 4 | 필수 | 11 | 페이지번호 |
| 한 페이지 결과 수 | numOfRows | 4 | 필수 | 10 | 한 페이지 결과 수 |
| 충전소 주소 | addr | 255 | 옵션 | 전라남도 나주시 전력로 55 | 검색대상 충전소주소 |

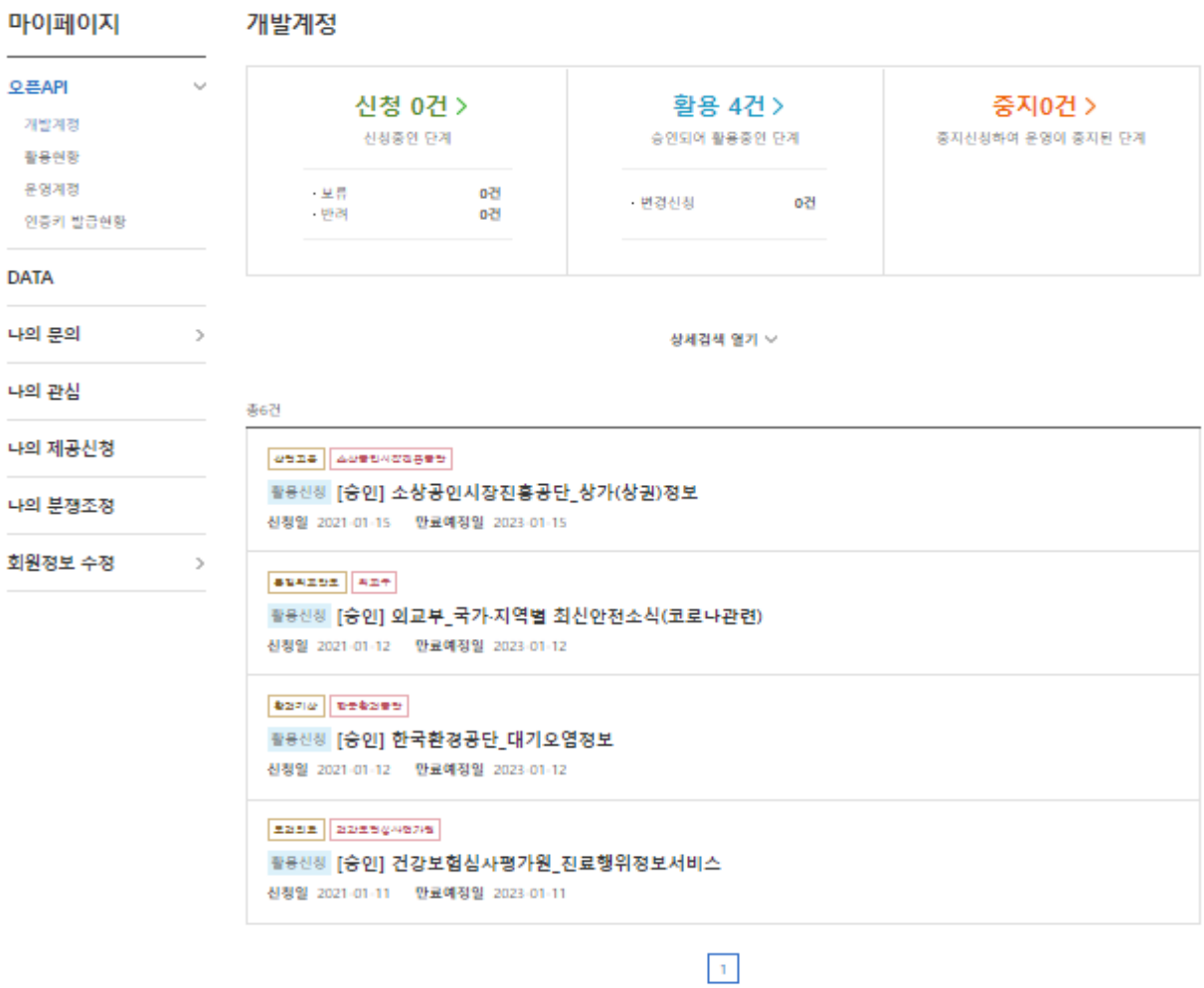
출력결과(Response Element)

| 항목명(국문) | 항목명(영문) | 항목크기 | 항목구분 | 샘플데이터 | 항목설명 |
|---|---|---|---|---|---|
| 결과코드 | resultCode | 2 | 필수 | | 결과코드 |
| 결과메시지 | resultMsg | 50 | 필수 | | 결과메시지 |
| 한 페이지 결과 수 | numOfRows | 4 | 필수 | 10 | 한 페이지 결과 수 |
| 페이지 번호 | pageNo | 4 | 필수 | 1 | 페이지번호 |
| 전체 결과 수 | totalCount | 4 | 필수 | 8 | 전체 결과 수 |
| 충전기주소 | addr | 255 | 필수 | 전라남도 나주시 전력로 55 | 충전소주소 |
| 충전기타입 | chargeTp | 1 | 필수 | 2 | 1 : 완속 2 : 급속 |
| 충전기ID | cpId | 12 | 필수 | 13 | 충전기ID |
| 충전기명칭 | cpNm | 20 | 필수 | 급속01 | 충전기명칭 |
| 충전기상태코드 | cpStat | 2 | 필수 | 1 | 1 : 충전가능 2: 충전중 3: 고장/점검 4 : 통신장애 5 : 통신미연결 |
| 충전방식 | cpTp | 2 | 필수 | 8 | 1 : B타입(5핀) 2 : C타입(5핀) 3 : BC타입(5핀) 4 : BC타입(7핀) 5 : DC차데모 6 : AC3상 7 : DC콤보 8 : DC차데모+DC콤보 |
| 충전소ID | csId | 12 | 필수 | 0 | 충전소ID |
| 충전소 명칭 | csNm | 50 | 필수 | 본사(나주) | 충전소 명칭 |
| 위도 | lat | 20 | 필수 | 35.02636975 | 위도 |
| 경도 | longi | 20 | 필수 | 126.7844551 | 경도 |
| 충전기 상태 갱신 시각 | statUpdatetime | 20 | 필수 | 2017-02-22 13:02:22 | 충전기 상태 갱신 시각 |

요청변수는 공공데이터 서버에 데이터를 보낼 때 인수를 의미하고 출력결과는 응답한 데이터를 의미한다. 응답한 데이터를 기준으로 자바에서 데이터를 클래스를 설계하면 된다.




활용목적을 작성하고 활용신청을 하면 바로 인증키가 발행되고 1~2 시간 이후에 활용이 가능함.

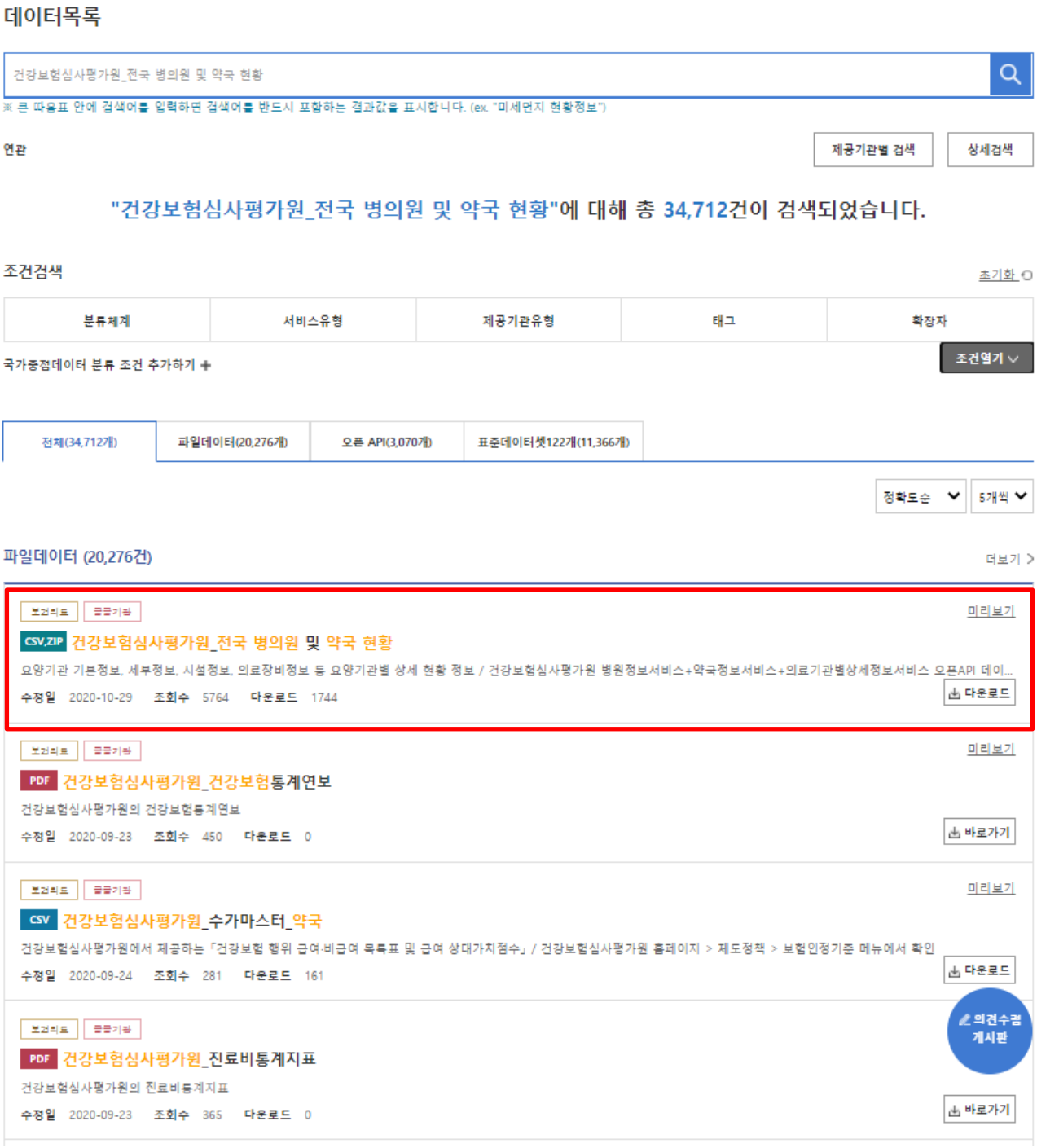



# 4 공공데이터 파일 java클래스로 저장하기

csv파일로 제공하는 데이터를 java애플리케이션에서 IO를 이용해서 활용하기 파일을 객체로 저장해서 처리하기하는 실습을 진행해보자.

## 4.1 공공데이터 구조 분석 후 클래스 설계하기

데이터 선정하기 공공데이터에 접속해서 "건강보험 심사평가원_전국 병의원 및 약국현황" 조회하면 관련 데이터가 검색결과에 아래와 같이 나오게 된다.




첫번째 데이터 선택하여 상세화면으로 이동하면 제공하는 데이터를 받을 수 있다. zip파일로 제공하고 있으며 그안에 있는 파일 중 csv파일을 불러와서 활용할 것이다.

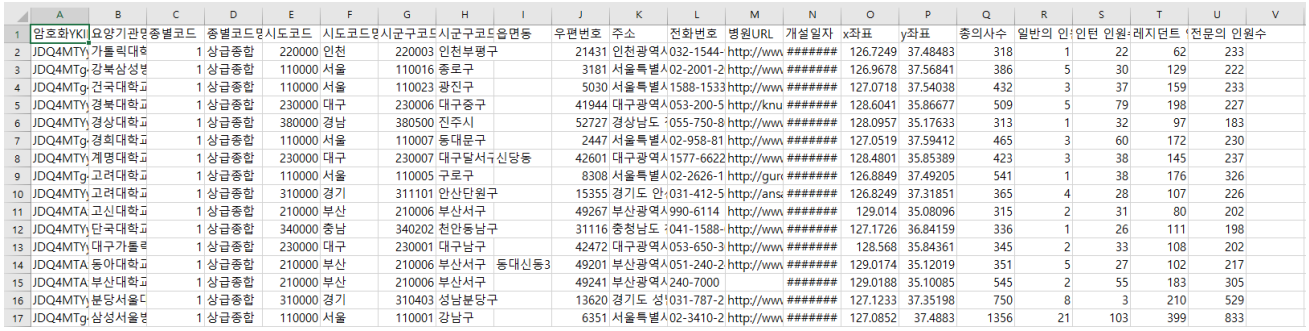



zip파일이 다운로드되면 내부에 여러파일이 있는데 그 중에서 병원정보서비스와 약국정보서비스를 이용해 보도록하겠다.

1. 병원정보서비스 2020.3Q.csv
2. 약국정보서비스 2020.3Q.csv
3. 의료기관별상세정보서비스(시설정보) 2020.3Q.csv
4. 의료기관별상세정보서비스(세부정보) 2020.3Q.csv
5. 의료기관별상세정보서비스(진료과목정보) 2020.3Q.csv
6. 의료기관별상세정보서비스(교통정보) 2020.3Q.csv
7. 의료기관별상세정보서비스(의료장비정보) 2020.3Q.csv
8. 의료기관별상세정보서비스(식대가산정보) 2020.3Q.csv
9. 의료기관별상세정보서비스(간호등급정보) 2020.3Q.csv
10. 의료기관별상세정보서비스(특수진료정보) 2020.3Q.csv
11. 의료기관별상세정보서비스(전문병원지정분야) 2020.3Q.csv

먼저 csv형식의 데이터를 이용하려면 파일의 내용이 어떠한 형식으로 되어있는지 구조를 분석해야한다. 먼저 병원정보서비스 파일을 열어보자. csv파일을 열 때 기본적으로 csv는 ‘,’와 ₩n으로 데이터를 구별하여 저장을 하게 되어있는데 excel프로그램이 설치되어있는 컴퓨터에서는 엑셀로 열려서 그 구조를 파알할 수 없기 때문에 파일에서 우클릭하고 편집을 눌러 메모장으로 열수있도록 한다.

* 엑셀로 open한 경우



| | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 암호화YKI | 요양기관명 | 종별코드 | 종별코드명 | 시도코드 | 시도코드명 | 시군구코드 | 시군구코드명 | 읍면동 | 우편번호 | 주소 | 전화번호 | 병원URL | 개설일자 | x좌표 | y좌표 | 총의사수 | 일반의 인원 | 인턴 인원 | 레지던트 인원 | 전문의 인원수 | |
| 2 | JDQ4MTy | 가톨릭대학 | 1 | 상급종합 | 220000 | 인천 | 220003 | 인천부평구 | | 21431 | 인천광역시 | 032-1544- | http://www | ######## | 126.7249 | 37.48483 | 318 | 1 | 22 | 62 | 233 | |
| 3 | JDQ4MTg | 강북삼성병 | 1 | 상급종합 | 110000 | 서울 | 110016 | 종로구 | | 3181 | 서울특별시 | 02-2001-2 | http://www | ######## | 126.9678 | 37.56841 | 386 | 5 | 30 | 129 | 222 | |
| 4 | JDQ4MTg | 건국대학교 | 1 | 상급종합 | 110000 | 서울 | 110023 | 광진구 | | 5030 | 서울특별시 | 1588-1533 | http://www | ######## | 127.0718 | 37.54038 | 432 | 3 | 37 | 159 | 233 | |
| 5 | JDQ4MTy | 경북대학교 | 1 | 상급종합 | 230000 | 대구 | 230006 | 대구중구 | | 41944 | 대구광역시 | 053-200-5 | http://knu | ######## | 128.6041 | 35.86677 | 509 | 5 | 79 | 198 | 227 | |
| 6 | JDQ4MTy | 경상대학교 | 1 | 상급종합 | 380000 | 경남 | 380500 | 진주시 | | 52727 | 경상남도 | 055-750-8 | http://www | ######## | 128.0957 | 35.17633 | 313 | 1 | 32 | 97 | 183 | |
| 7 | JDQ4MTg | 경희대학교 | 1 | 상급종합 | 110000 | 서울 | 110007 | 동대문구 | | 2447 | 서울특별시 | 02-958-81 | http://www | ######## | 127.0519 | 37.59412 | 465 | 3 | 60 | 172 | 230 | |
| 8 | JDQ4MTy | 계명대학교 | 1 | 상급종합 | 230000 | 대구 | 230007 | 대구달서구 | 신당동 | 42601 | 대구광역시 | 1577-6622 | http://www | ######## | 128.4801 | 35.85389 | 423 | 3 | 38 | 145 | 237 | |
| 9 | JDQ4MTg | 고려대학교 | 1 | 상급종합 | 110000 | 서울 | 110005 | 구로구 | | 8308 | 서울특별시 | 02-2626-1 | http://guro | ######## | 126.8849 | 37.49205 | 541 | 1 | 38 | 176 | 326 | |
| 10 | JDQ4MTy | 고려대학교 | 1 | 상급종합 | 310000 | 경기 | 311101 | 안산단원구 | | 15355 | 경기도 안 | 031-412-5 | http://ansa | ######## | 126.8249 | 37.31851 | 365 | 4 | 28 | 107 | 226 | |
| 11 | JDQ4MTA | 고신대학교 | 1 | 상급종합 | 210000 | 부산 | 210006 | 부산서구 | | 49267 | 부산광역시 | 990-6114 | http://www | ######## | 129.014 | 35.08096 | 315 | 2 | 31 | 80 | 202 | |
| 12 | JDQ4MTy | 단국대학교 | 1 | 상급종합 | 340000 | 충남 | 340202 | 천안동남구 | | 31116 | 충청남도 | 041-1588- | http://www | ######## | 127.1726 | 36.84159 | 336 | 1 | 26 | 111 | 198 | |
| 13 | JDQ4MTy | 대구가톨릭 | 1 | 상급종합 | 230000 | 대구 | 230001 | 대구남구 | | 42472 | 대구광역시 | 053-650-3 | http://www | ######## | 128.568 | 35.84361 | 345 | 2 | 33 | 108 | 202 | |
| 14 | JDQ4MTA | 동아대학교 | 1 | 상급종합 | 210000 | 부산 | 210006 | 부산서구 | 동대신동3 | 49201 | 부산광역시 | 051-240-2 | http://www | ######## | 129.0174 | 35.12019 | 351 | 5 | 27 | 102 | 217 | |
| 15 | JDQ4MTA | 부산대학교 | 1 | 상급종합 | 210000 | 부산 | 210006 | 부산서구 | | 49241 | 부산광역시 | 240-7000 | | ######## | 129.0188 | 35.10085 | 545 | 2 | 55 | 183 | 305 | |
| 16 | JDQ4MTy | 분당서울대 | 1 | 상급종합 | 310000 | 경기 | 310403 | 성남분당구 | | 13620 | 경기도 성 | 031-787-2 | http://www | ######## | 127.1233 | 37.35198 | 750 | 8 | 3 | 210 | 529 | |
| 17 | JDQ4MTg | 삼성서울병 | 1 | 상급종합 | 110000 | 서울 | 110001 | 강남구 | | 6351 | 서울특별시 | 02-3410-2 | http://www | ######## | 127.0852 | 37.4883 | 1356 | 21 | 103 | 399 | 833 | |

* 편집으로 open한 경우



1. 병원정보서비스 2020.3Q.csv - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

"암호화YKIHO코드","요양기관명","종별코드","종별코드명","시도코드","시도코드명","시군구코드","시군구코드명","읍면동","우편번호","주소","전화번호","병원URL","개설일자","x좌표","y좌표","총의사수","일반의 인원수","인턴 인원수","레지던트 인원수","전문의 인원수"

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjE4MzIjNDEjIDEjIDgjIDgz","가톨릭대학교인천성모병원","1","상급종합","220000","인천","220003","인천부평구","","21431","인천광역시 부평구 동수로 56 (부평동)","032-1544-9004","http://www.cmcism.or.kr/","1981-08-06","126.7248987","37.4848309","318","1","22","62

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQzNjE4MDIjNjEjIDEjIDQjIDgz","강북삼성병원","1","상급종합","110000","서울","110016","종로구","","03181","서울특별시 종로구 새문안로 29 (평동)","02-2001-2001","http://www.kbsmc.co.kr","1979-03-24","126.96775","37.5684083","386","5","30","129","222

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ2MiQyNjE4MzIjNjEjIDEjIDQjIDgz","건국대학교병원","1","상급종합","110000","서울","110023","광진구","","05030","서울특별시 광진구 능동로 120-1 (화양동)","1588-1533","http://www.kuh.ac.kr","1982-11-16","127.0718276","37.5403764","432","3","37","159","233

"JDQ4MTyyMiM4MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjEwMDIjNzEjIDEjIDgjIDgz","경북대학교병원","1","상급종합","230000","대구","230006","대구중구","","41944","대구광역시 중구 동덕로 130 (삼덕동2가, 경북대학교병원)","053-200-5114","http://knumc.knu.ac.kr","1910-09-07","128.6041","35.8667744","509","5","79","1

"JDQ4MTyyMiM4MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ5OSQyNjEwMDIjNjEjIDEjIDQjIDgz","경상대학교병원","1","상급종합","380000","경남","380500","진주시","","52727","경상남도 진주시 강남로 79 (칠암동)","055-750-8000","http://www.gnuh.co.kr","1986-10-13","128.0956717","35.1763252","313","1","32","97","183

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ4OSQzNjEyMjIjNjEjIDEjIDQjIDgz","경희대학교병원","1","상급종합","110000","서울","110007","동대문구","","02447","서울특별시 동대문구 경희대로 23 (회기동)","02-958-8114","http://www.khuh.or.kr/","1971-10-05","127.051852","37.5941195","465","3","60","172","230

"JDQ4MTyyMiM4MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjEyMjIjNjEjIDEjIDQjIDgz","계명대학교동산병원","1","상급종합","230000","대구","230007","대구달서구","신당동","42601","대구광역시 달서구 달구벌대로 1035 (신당동)","1577-6622","http://www.dsmc.or.kr/","1968-04-06","128.4801315","35.8538856","423","3","38","14

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQzMiQ0NjEwMDIjNTEjIDEjIDQjIDAjIDgz","고려대학교의과대학부속구로병원","1","상급종합","110000","서울","110005","구로구","","08308","서울특별시 구로구 구로동로 148 고려대부속구로병원 (구로동)","02-2626-1114","http://guro.kumc.or.kr/main/index.do","1983-08-31","126.884

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ5OSQzNjEwMDIjNTEjIDEjIDQjIDgz","고려대학교의과대학부속안산병원","1","상급종합","310000","경기","311101","안산단원구","","15355","경기도 안산시 단원구 적금로 123 (고잔동)","031-412-5653","http://ansan.kumc.or.kr/main/index.do","1986-01-01","126.8249033","37.3185

"JDQ4MTAxMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjE4MzIjODEjIDEjIDYjIDgz","고신대학교복음병원","1","상급종합","210000","부산","210006","부산서구","","49267","부산광역시 서구 감천로 262 (암남동)","990-6114","http://www.kosinmed.or.kr/","1974-02-26","129.0139595","35.0809636","315","2","31","80","202

"JDQ4MTyyMiM2MSMkMSMkMiMkODkkMzgxMzUxIzExIyQxIyQzIyQ4OSQ0NjEwMDIjODEjIDEjIDYjIDgz","단국대학교의과대학부속병원","1","상급종합","340000","충남","340202","천안동남구","","31116","충청남도 천안시 동남구 망향로 201 (안서동)","041-1588-0063","http://www.dkuh.co.kr/main/default.asp","1994-04-25","127.172575","36.8415

"JDQ4MTyyMiM4MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQzNjEyMjIjNjEjIDEjIDQjIDgz","대구가톨릭대학교병원","1","상급종합","230000","대구","230001","대구남구","","42472","대구광역시 남구 두류공원로17길 33 (대명동)","053-650-3000","http://www.dcmc.co.kr/","1980-08-12","128.5679969","35.8436078","345","2","33","108

"JDQ4MTAxMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQwMCQ0NjEwMDIjNTEjIDEjIDQjIDgz","동아대학교병원","1","상급종합","210000","부산","210006","부산서구","동대신동3가","49201","부산광역시 서구 대신공원로 26 (동대신동3가)","051-240-2400","http://www.damc.or.kr","1990-03-03","129.017425","35.12019","351","5","27","

"JDQ4MTAxMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjEyMjIjNDEjIDEjIDgjIDgz","부산대학교병원","1","상급종합","210000","부산","210006","부산서구","","49241","부산광역시 서구 구덕로 179 (아미동1가)","240-7000","","1981-08-27","129.0188032","35.1008477","545","2","55","183","305

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQwMyQyNjEwMDIjNTEjIDEjIDYjIDgz","분당서울대학교병원","1","상급종합","310000","경기","310403","성남분당구","","13620","경기도 성남시 분당구 구미로173번길 82 (구미동, 분당서울대학교병원)","031-787-2114","http://www.snubh.org/index.do","2003-04-15","127.1232509","

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQxMyQzNjEwMDIjODEjIDEjIDIjIDgz","삼성서울병원","1","상급종합","110000","서울","110001","강남구","","06351","서울특별시 강남구 일원로 81 (일원동, 삼성의료원)","02-3410-2114","http://www.samsunghospital.com","1994-06-13","127.0851508","37.4882978","1356","21","1

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQzNjE4MzIjODEjIDEjIDYjIDgz","서울대학교병원","1","상급종합","110000","서울","110016","종로구","","03080","서울특별시 종로구 대학로 101 (연건동)","1588-5700","http://www.snuh.org","1981-07-09","126.9990168","37.5797151","1586","15","180","565","826

"JDQ4MTyyMiM2MSMkMSMkMiMkODkkMzgxMzUxIzExIyQxIyQzIyQ5OSQ0NjEwMDIjNjEjIDEjIDgjIDgz","순천향대학교부속 천안병원","1","상급종합","340000","충남","340202","천안동남구","","31151","충청남도 천안시 동남구 순천향6길 31 5동 (봉정동,순천향대학교병원)","041-570-2114","http://www.schmc.ac.kr/cheonan/kor/index.do","1982-06-

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ5MiQzNjEyMjIjNzEjIDEjIDgjIDgz","순천향대학교부속부천병원","1","상급종합","310000","경기","310300","부천시","","14584","경기도 부천시 조마루로 170 (중동)","032-621-5114","http://www.schmc.ac.kr/bucheon/kor/index.do","2001-01-29","126.7629514","37.4986043","363",

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ2MiQzNjE4MzIjNTEjIDEjIDYjIDgz","아주대학교병원","1","상급종합","310000","경기","310604","수원영통구","","16499","경기도 수원시 영통구 월드컵로 164 (원천동)","031-219-5114","http://hosp.ajoumc.or.kr/","1994-04-25","127.0476764","37.2793927","676","18","63","195","4

"JDQ4MTyyMiM4MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQzMiQyNjE0ODEjODEjIDEjIDYjIDgz","양산부산대학교병원","1","상급종합","380000","경남","381100","양산시","","50612","경상남도 양산시 물금읍 금오로 20 0","055-360-1000","https://www.pnuyh.or.kr","2008-10-24","129.0057274","35.3281501","438","11","29","110","298

"JDQ4MTyyMiM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQ3OSQyNjE4MzIjNjEjIDEjIDQjIDgz","연세대학교 원주세브란스기독병원","1","상급종합","320000","강원","320400","원주시","","26426","강원도 원주시 일산로 20-0 (일산동)","033-741-0114","http://wch.or.kr/","1977-05-01","127.9456639","37.3490013","434","15","43","138","238

"JDQ4MTg4MSM1MSMkMSMkMCMkODkkMzgxMzUxIzExIyQxIyQzIyQzMiQzNjE0ODEjNTEjIDEjIDYjIDgz","연세대학교의과대학 강남세브란스병원","1","상급종합","110000","서울","110001","강남구","","06273","서울특별시 강남구 언주로 211 강남세브란스병원 (도곡동)","02-2019-3114","http://gs.iseverance.com","1983-04-04","127.0462683","37.45

편집으로 open한 경우 각 라인별로 데이터가 들어가고 데이터를 구분한 것은 ₩n이고, 각 항목별로 구분은 ‘ , ’로 한 것을 볼수가 있다.

출력된 내용을 보고 애플리케이션에 필요한 내용을 선별하여 데이터를 보관할 객체를 설계할 수 있다. 병원 항목을 보면 "암호화YKIHO코드" … "전문의 인원수"까지 21개의 항목이 있는데 프로그램에서 필요한 정보만 선별하면 되겠다. 실습에서는 종별코드명, 기관명, 시도코명 시군구코드명, 주소, 전화번호, 병원URL 개설일, 총의사수만 활용해보도록 하겠다.

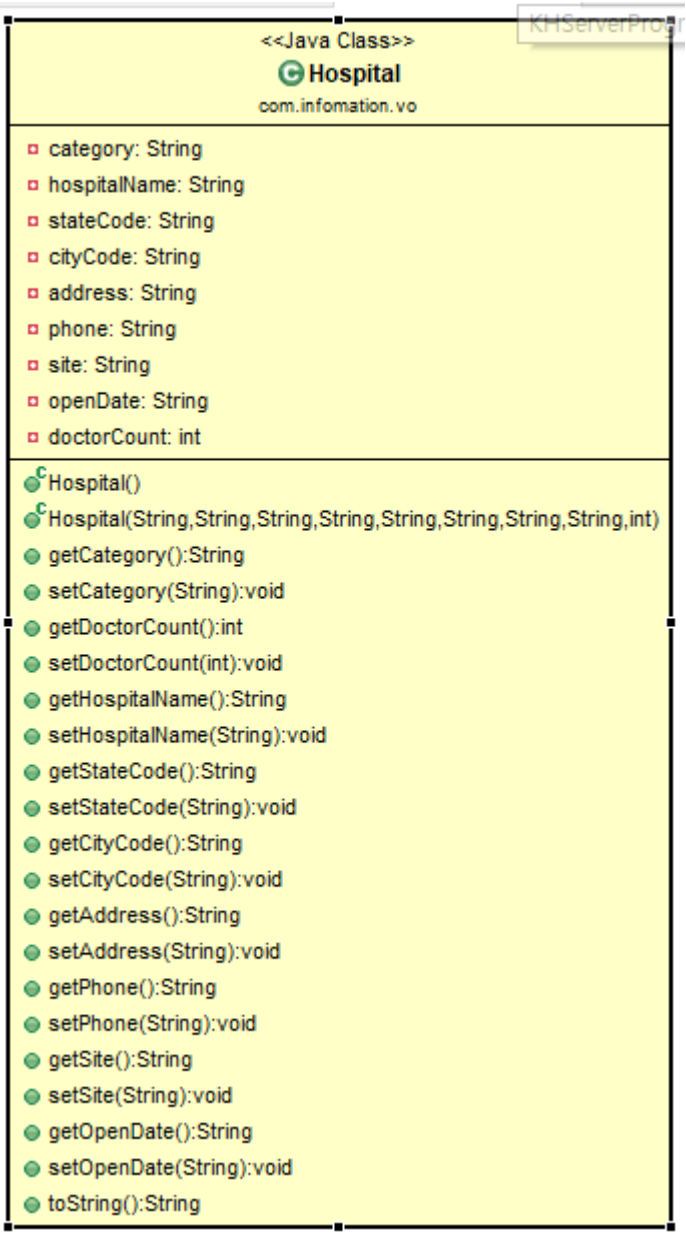
이렇게 필요항목을 선별하고 그 항목의 데이터들을 저장할 수 있는 클래스를 설계한다.

여기서는 데이터를 저장하는 저장용 클래스를 설계하는 것으로 데이터를 보관하는 필드(멤버변수)를 잘 설계해야한다. 필드를 설정할때는 들어오는 데이터가 무엇인지에 따라 자료형을 정하고 선언을 해야한다.

각 항복별 들어가 있는 데이터를 보면 알수가 있다. 의사수, 개설일자를 제외한 나머지 값들은 모두 문자열이 들어가 있고, 의사수는 정수, 개설일자는 날짜가 들어가 있는 구조이다. 이 구조에 맞춰 병원정보를 저장하는 클래스를 설계할 수 있다.

## Hospital클래스 구성하기

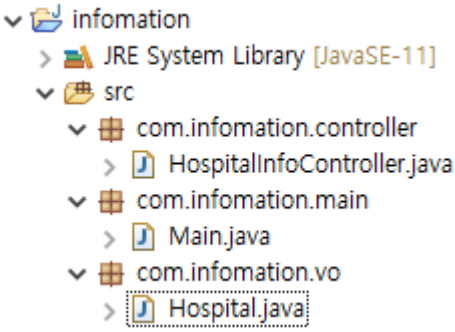
각 데이터에 맞게 자료형 및 변수명을 선정하여 클래스를 설계할 수 있다. 간단하게 csv에 있는 데이터를 저장할 수 있는 Hospital클래스를 설계하였다.




*** 더 정보가 필요하다면 다른 csv와 연결하여 클래스를 설계할 수도 있다.**

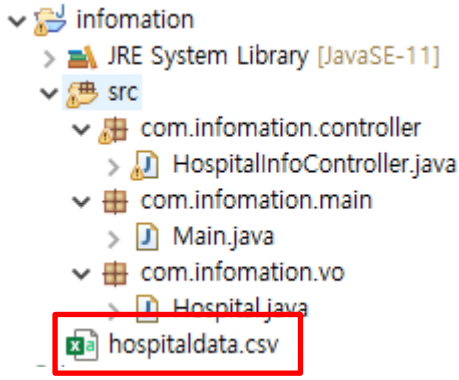
## 4.2 csv파일 프로젝트에 저장하기

프로젝트를 생성해서 공공데이터가 제공하는 병원정보를 설계된 클래스에 저장해보자. 프로젝트 구조




3개의 패키지를 생성하고 Main.java은 프로젝트를 실행해주는 클래스, HospitalInfoController.java Hospital객체를 관리하는 클래스, Hospital.java 병원정보를 저장하는 클래스로 나눴다.

csv파일이 데이터를 불러오기 위해 프로젝트 최상단에 파일을 저장시키자. 파일을 불러오기 편하게 하기 위해 파일명을 hospitalData.csv로변경하여 저장하자




## 4.3 csv파일 파일 java객체로 저장/활용하기

### Main.java 구성하기

Main.java클래스에서는 mian()매소드로 간단하게 csv파일에 있는 데이터를 java로 전환하는 메소드 호출과 변환 후 데이터를 출력하는 메소드 호출을 하겠다.

**com.infomation.main**

```
package com.infomation.main;

import com.infomation.controller.HospitalInfoController;

public class Main {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		HospitalInfoController controller=new
HospitalInfoController();//저장된 list값을 유지하기 위해 변수로 주소값을 가지고 있음
		controller.inputData();//프로젝트에 저장된 병원정보 csv파일을 가져와
java클래스에 대입하는 매소드(기능)
		controller.printHospitalData();//대입된 내용을 출력해주는 매소드(기능)
	}

}
```

## Hospital.java 구성하기

csv에서 가져오는 데이터를 저장하는 저장용 클래스로 csv파일 항목명에 맞는 데이터저장 변수를 가지고 있는 객체이다. 저장내용은 종류, 병원명, 시도, 시군구, 주소, 전화번호, 홈페이지, 개설일, 의사수이다. 개설일은 날짜형 자료지만 나중에 필요시 Date형으로 변환하고 기본저장은 String으로 저장하겠다.

com.infomation.vo.Hospital

```
package com.infomation.vo;

public class Hospital {

	private String category;//종별
	private String hospitalName;//요양기관명
	private String stateCode;//시도코드명
	private String cityCode;//시군구코드명
	private String address;//주소
	private String phone;//전화번호
	private String site;//홈페이지주소
	private String openDate;//개설일
	private int doctorCount;//총의사수

	public Hospital() {
		// TODO Auto-generated constructor stub
	}

	public Hospital(String category, String hospitalName, String stateCode,
String cityCode, String address,
			String phone, String site, String openDate, int doctorCount)
{
		super();
		this.category = category;
		this.hospitalName = hospitalName;
		this.stateCode = stateCode;
		this.cityCode = cityCode;
		this.address = address;
		this.phone = phone;
		this.site = site;
		this.openDate = openDate;
		this.doctorCount = doctorCount;
	}


	public String getCategory() {
		return category;
	}

	public void setCategory(String category) {
		this.category = category;
```

```
	}

	public int getDoctorCount() {
		return doctorCount;
	}

	public void setDoctorCount(int doctorCount) {
		this.doctorCount = doctorCount;
	}

	public String getHospitalName() {
		return hospitalName;
	}

	public void setHospitalName(String hospitalName) {
		this.hospitalName = hospitalName;
	}

	public String getStateCode() {
		return stateCode;
	}

	public void setStateCode(String stateCode) {
		this.stateCode = stateCode;
	}

	public String getCityCode() {
		return cityCode;
	}

	public void setCityCode(String cityCode) {
		this.cityCode = cityCode;
	}

	public String getAddress() {
		return address;
	}

	public void setAddress(String address) {
		this.address = address;
	}

	public String getPhone() {
		return phone;
	}

	public void setPhone(String phone) {
		this.phone = phone;
	}

	public String getSite() {
		return site;
	}

	public void setSite(String site) {
		this.site = site;
	}
```

```
	public String getOpenDate() {
		return openDate;
	}

	public void setOpenDate(String openDate) {
		this.openDate = openDate;
	}


	@Override
	public String toString() {
		return "Hospital [hospitalName=" + hospitalName + ", stateCode=" +
stateCode + ", cityCode=" + cityCode
				+ ", address=" + address + ", phone=" + phone + ",
site=" + site + ", openDate=" + openDate
				+ ", docktorCount=" + doctorCount + "]";
	}
```

## HospitalInfoController.java 구성하기

Csv에 있는 데이터를 저장할 수 있는 List<Hospital>을 필드(멤버변수)로 선언하고, csv에서 데이터를 가져와 Hospital객체로 파싱해주는 inputData()매소드를 구현하고, List<Hospital>에 저장된 내용을 출력하는 `printHospitalData()매소드를 구현하겠다.`

### com.infomation.controller.HospitalInfoController

```
package com.infomation.controller;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.StringTokenizer;

import com.infomation.vo.Hospital;


public class HospitalInfoController {

	private List<Hospital> list=new ArrayList();

	public void inputData() {
		//csv파일 불러오기
		String path=HospitalInfoController.class.getResource("/").getPath();
		File data=new File(path+"/hospitaldata.csv");
		StringBuffer sb=new StringBuffer();
		//문자열로 저장되거있어 Reader를 이용하고 받아오는 속도를 향상하기 위해
BufferedReader보조 스트림을 사용하였다.
		//각데이터를 라인별로 받아서 StringBuffer에 저장하여 관리함.
		try(BufferedReader breader=new BufferedReader(new
FileReader(data));){
			String ch="";
```

```
                        while((ch=breader.readLine())!=null) {
                                sb.append(ch+"\n");//csv에서 받아온파일을
StringBuffer에 저장
                        }

                }catch(IOException e) {
                        e.printStackTrace();
                }

                String[] datas=sb.toString().split("\n");//버퍼에 저장된 내용을
라인별로 나눠거 배열로 저장

                for(String d : datas) {

                        String[]
detail=d.replace("\",","@").replace("\"","").split("@");//항목별로 다시 배열로
저장하여 각 데이터 불러오기
                        //csv에서 사용한 ,가 주소를 표시하는 부분에도 있어 항목을 구분한 ,
를 @로 변경하여 파싱하고, 데이터 부분은 모두 ""이 붙어있어 이를 제거하는 구문이다.

                        //원하는 항목만 hospital객체에 대입하여 저장하기.
                        int count=0;//csv의 최초라인이 항목명을 가져오고있어 그부분에
대한 예외처리 구문
                        try {
                                count=Integer.parseInt(detail[16]);
                        }catch(NumberFormatException e) {

                        }
                        //모든 항목데이터가 배열로 대입이 되어있어, 배열의 인덱스로
해당하는 자료를 대입하는 구문
                        list.add(new
Hospital(detail[3],detail[1],detail[5],detail[7],detail[10],detail[11],detail[12],
detail[13],count));

                }
                list.remove(0);//항목명 부분 삭제
        }

        public void printHospitalData() {
                System.out.printf("%-10s %-20s %-10s %-10s %-30s %-10s %-30s %-
10s %-4s\n",

"종류","병원명","시도명","시군구명","주소","전화번호","홈페이지주소","개설일","총의사수"
);//출력시 출력자리를 맞춰주기 위해 printf를 사용하였음.

                for(Hospital h : list) {
                        System.out.printf("%-10s %-20s %-10s %-10s %-30s %-10s %-
30s %-10s %-4d\n",

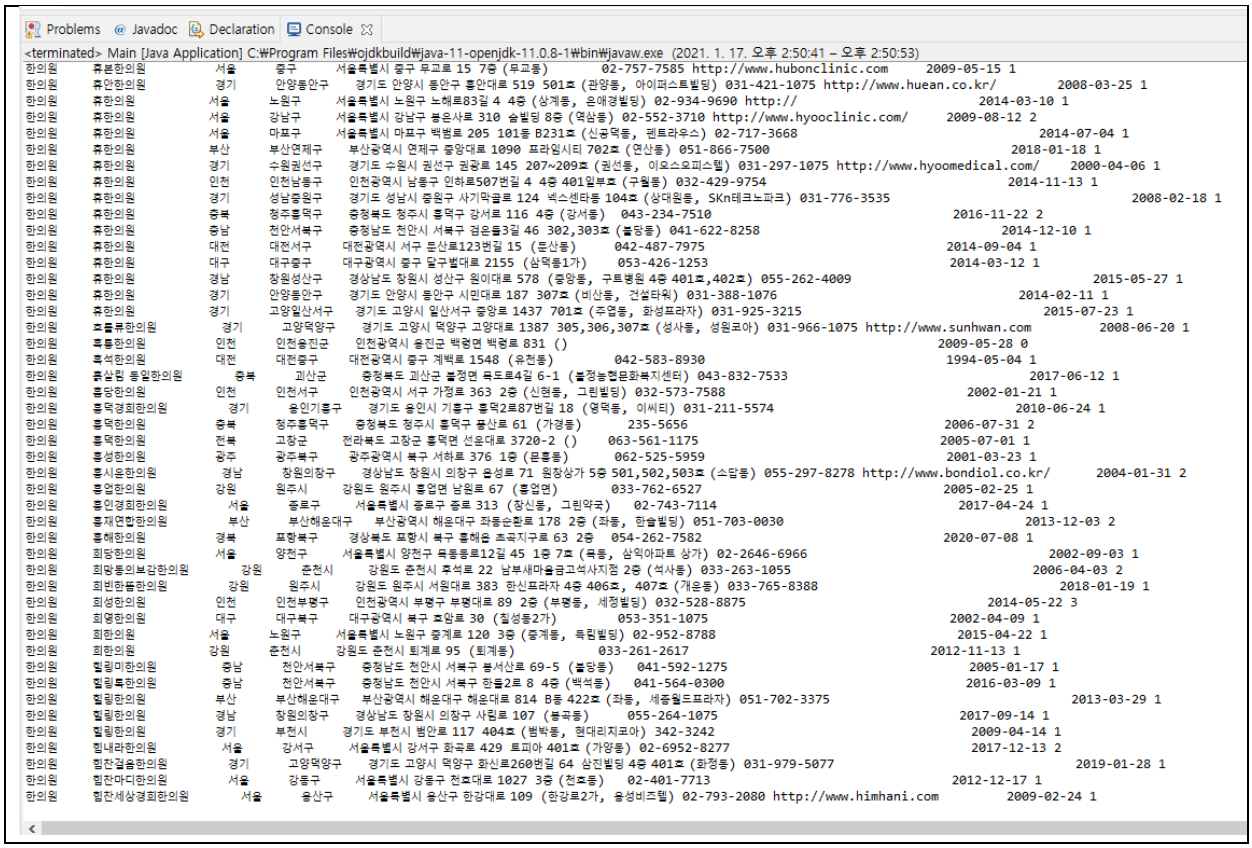
h.getCategory(),h.getHospitalName(),h.getStateCode(),h.getCityCode(),h.getAddress(
),h.getPhone(),h.getSite(),h.getOpenDate(),h.getDoctorCount());
```

```
                }
        }



}
```

## 실행결과



Problems Javadoc Declaration Console

<terminated> Main [Java Application] C:₩Program Files₩ojdkbuild₩java-11-openjdk-11.0.8-1₩bin₩javaw.exe (2021. 1. 17. 오후 2:50:41 – 오후 2:50:53)

```
한의원  휴본한의원  서울  중구  서울특별시 중구 무교로 15 7층 (무교동)  02-757-7585 http://www.hubonclinic.com  2009-05-15 1
한의원  휴안한의원  경기  안양동안구  경기도 안양시 동안구 흥안대로 519 501호 (관양동, 아이퍼스트빌딩) 031-421-1075 http://www.huean.co.kr/  2008-03-25 1
한의원  휴한의원  서울  노원구  서울특별시 노원구 노해로83길 4 4층 (상계동, 은애경빌딩) 02-934-9690 http://  2014-03-10 1
한의원  휴한의원  서울  강남구  서울특별시 강남구 봉은사로 310 슬빌딩 8층 (역삼동) 02-552-3710 http://www.hyooclinic.com/  2009-08-12 2
한의원  휴한의원  서울  마포구  서울특별시 마포구 백범로 205 101동 B231호 (신공덕동, 펜트라우스) 02-717-3668  2014-07-04 1
한의원  휴한의원  부산  부산연제구  부산광역시 연제구 중앙대로 1090 프라임시티 702호 (연산동) 051-866-7500  2018-01-18 1
한의원  휴한의원  경기  수원권선구  경기도 수원시 권선구 권광로 145 207~209호 (권선동, 이오스오피스텔) 031-297-1075 http://www.hyoomedical.com/  2000-04-06 1
한의원  휴한의원  인천  인천남동구  인천광역시 남동구 인하로507번길 4 4층 401일부호 (구월동) 032-429-9754  2014-11-13 1
한의원  휴한의원  경기  성남중원구  경기도 성남시 중원구 사기막골로 124 넥스센타동 104호 (상대원동, SKn테크노파크) 031-776-3535  2008-02-18 1
한의원  휴한의원  충북  청주흥덕구  충청북도 청주시 흥덕구 강서로 116 4층 (강서동)  043-234-7510  2016-11-22 2
한의원  휴한의원  충남  천안서북구  충청남도 천안시 서북구 검은들3길 46 302,303호 (불당동) 041-622-8258  2014-12-10 1
한의원  휴한의원  대전  대전서구  대전광역시 서구 둔산로123번길 15 (둔산동)  042-487-7975  2014-09-04 1
한의원  휴한의원  대구  대구중구  대구광역시 중구 달구벌대로 2155 (삼덕동1가)  053-426-1253  2014-03-12 1
한의원  휴한의원  경남  창원성산구  경상남도 창원시 성산구 원이대로 578 (중앙동, 구트병원 4층 401호,402호) 055-262-4009  2015-05-27 1
한의원  휴한의원  경기  안양동안구  경기도 안양시 동안구 시민대로 187 307호 (비산동, 건설타워) 031-388-1076  2014-02-11 1
한의원  휴한의원  경기  고양일산서구  경기도 고양시 일산서구 중앙로 1437 701호 (주엽동, 화성프라자) 031-925-3215  2015-07-23 1
한의원  흐름류한의원  경기  고양덕양구  경기도 고양시 덕양구 고양대로 1387 305,306,307호 (성사동, 성원로아) 031-966-1075 http://www.sunhwan.com  2008-06-20 1
한의원  흑룡한의원  인천  인천송진군  인천광역시 송진군 백령면 백령로 831 ()  2009-05-28 0
한의원  흑석한의원  대전  대전중구  대전광역시 중구 계백로 1548 (유천동)  042-583-8930  1994-05-04 1
한의원  흙살림 통일한의원  충북  괴산군  충청북도 괴산군 불정면 육도로4길 6-1 (불정농협문화복지센터) 043-832-7533  2017-06-12 1
한의원  흥당한의원  인천  인천서구  인천광역시 서구 가정로 363 2층 (신현동, 그린빌딩) 032-573-7588  2002-01-21 1
한의원  흥덕경희한의원  경기  용인기흥구  경기도 용인시 기흥구 흥덕2로87번길 18 (영덕동, 이씨티) 031-211-5574  2010-06-24 1
한의원  흥덕한의원  충북  청주흥덕구  충청북도 청주시 흥덕구 풍산로 61 (가경동)  235-5656  2006-07-31 2
한의원  흥덕한의원  전북  고창군  전라북도 고창군 흥덕면 선운대로 3720-2 ()  063-561-1175  2005-07-01 1
한의원  흥성한의원  광주  광주북구  광주광역시 북구 서하로 376 1층 (문흥동)  062-525-5959  2001-03-23 1
한의원  흥시온한의원  경남  창원의창구  경상남도 창원시 의창구 중성로 71 원창상가 5층 501,502,503호 (소답동) 055-297-8278 http://www.bondiol.co.kr/  2004-01-31 2
한의원  흥업한의원  강원  원주시  강원도 원주시 흥업면 남원로 67 (흥업면)  033-762-6527  2005-02-25 1
한의원  흥인경희한의원  서울  종로구  서울특별시 종로구 종로 313 (창신동, 그린약국)  02-743-7114  2017-04-24 1
한의원  흥재연합한의원  부산  부산해운대구  부산광역시 해운대구 좌동순환로 178 2층 (좌동, 한솔빌딩) 051-703-0030  2013-12-03 2
한의원  흥해한의원  경북  포항북구  경상북도 포항시 북구 흥해읍 초곡지구로 63 2층  054-262-7582  2020-07-08 1
한의원  희당한의원  서울  양천구  서울특별시 양천구 목동동로12길 45 1층 7호 (목동, 삼익아파트 상가) 02-2646-6966  2002-09-03 1
한의원  희망동의보감한의원  강원  춘천시  강원도 춘천시 후석로 22 남부새마을금고석사지점 2층 (석사동) 033-263-1055  2006-04-03 2
한의원  희빈한뜰한의원  강원  원주시  강원도 원주시 서원대로 383 한신프라자 4층 406호, 407호 (개운동) 033-765-8388  2018-01-19 1
한의원  희성한의원  인천  인천부평구  인천광역시 부평구 부평대로 89 2층 (부평동, 세정빌딩) 032-528-8875  2014-05-22 3
한의원  희영한의원  대구  대구북구  대구광역시 북구 호암로 30 (칠성동2가)  053-351-1075  2002-04-09 1
한의원  희한의원  서울  노원구  서울특별시 노원구 중계로 120 3층 (중계동, 특림빌딩) 02-952-8788  2015-04-22 1
한의원  희한의원  강원  춘천시  강원도 춘천시 퇴계로 95 (퇴계동)  033-261-2617  2012-11-13 1
한의원  헬링미한의원  충남  천안서북구  충청남도 천안시 서북구 봉서산로 69-5 (불당동)  041-592-1275  2005-01-17 1
한의원  헬링목한의원  충남  천안서북구  충청남도 천안시 서북구 한들2로 8 4층 (백석동)  041-564-0300  2016-03-09 1
한의원  헬링한의원  부산  부산해운대구  부산광역시 해운대구 해운대로 814 B동 422호 (좌동, 세종월드프라자) 051-702-3375  2013-03-29 1
한의원  헬링한의원  경남  창원의창구  경상남도 창원시 의창구 사림로 107 (봉곡동)  055-264-1075  2017-09-14 1
한의원  헬링한의원  경기  부천시  경기도 부천시 범안로 117 404호 (범박동, 현대리치코아) 342-3242  2009-04-14 1
한의원  헬내라한의원  서울  강서구  서울특별시 강서구 화곡로 429 토피아 401호 (가양동) 02-6952-8277  2017-12-13 2
한의원  헬찬걸음한의원  경기  고양덕양구  경기도 고양시 덕양구 화신로260번길 64 삼진빌딩 4층 401호 (화정동) 031-979-5077  2019-01-28 1
한의원  헬찬마디한의원  서울  강동구  서울특별시 강동구 천호대로 1027 3층 (천호동)  02-401-7713  2012-12-17 1
한의원  헬찬세상경희한의원  서울  송산구  서울특별시 송산구 한강대로 109 (한강로2가, 용성비즈텔) 02-793-2080 http://www.himhani.com  2009-02-24 1
```

console창에 출력된 결과로 7만건이나 되는 데이터를 공공데이터에서 제공하는 파일을 이용하여 손쉽게 애플리케이션에 적용할 수 있다. 향후 이 데이터를 항목에 따라 검색하는 기능이나 추가적인 병원내용의 csv를 가져와 더 폭넓은 애플리케이션을 구현할 수 있다.

지금가지 공공포털에서 제공하는 파일데이터를 이용하여 애플리케이션을 만드는 방법을 배워보았다.

# Appendix. 이클립스 단축키 사용법

**단축키 확인방법 : Preference > General > Keys** Ctrl+Shift+L : 단축키 보기 Hint

# 1 자주 사용하는 단축키

**ctrl + s** : 저장 및 컴파일
**ctrl + i** : 소스 깔끔 정리(인덴트 중심의 자동구문정리)
**ctrl + space** : 어휘의 자동완성(Content Assistance)
**ctrl + 1** : Quick Fix(Rename 에 주로 사용)
**ctrl + shift + M** : 캐럿이 위치한 대상에 필요한 특정클래스 import
**ctrl + shift + O** : 소스에 필요한 패키지의 자동 임포트
**ctrl + /** : 한줄 또는 선택영역 주석처리/제거
**ctrl + Q** : 마지막 편집위치로 가기
**ctrl + L** : 특정줄번호로 가기
**ctrl + D** : 한줄삭제
**ctrl + F** : Find 및 Replace
**ctrl + K** : 다음찾기(또는, 찾고자 하는 문자열을 블럭으로 설정한 후 키를 누른다.)
**ctrl + shift + K** : 이전찾기(또는, 찾고자 하는 문자열을 블럭으로 설정한 후 역으로 찾고자 하는 문자열을 찾아감.)
**alt + shift + j** : 설정해 둔 기본주석 달기
**Ctrl + 객체클릭(혹은 F3)** : 클래스나 메소드 혹은 멤버를 정의한 곳으로 이동(Open Declaration)

# 2 유용한 단축키

**ctrl + shift + f** : 소스 깔끔 정리
**ctrl + shift + /** : 선택영역 block comment 설정
**ctrl + shift + ₩** : 선택영역 block comment 제거
**alt + shift + up** : Enclosing Element 선택(괄호의 열고 닫기 쌍 확인에 유용함)
**ctrl + O** : Outline 창열기
**Alt + ->, Alt + <-** : 이후, 이전
**해당프로젝트에서 alt + enter** : Project 속성
**sysout > Ctrl + Space** : System.out.println();
**try > Ctrl + Space** : 기본 try-catch 문 완성
**for > Ctrl + Space** : 기본 for 문 완성

# 3 기타 단축키

**ctrl + N** : 새로운 파일 및 프로젝트 생성
**ctrl + shift + s** : 열려진 모든파일 저장 및 컴파일
**alt + /** : Word Completion
**alt + shift + R** : Rename
**ctrl + shift + G** : 특정 메소드나 필드를 참조하고 있는 곳을 찾는다.
**ctrl + shift + B** : 현재커서위치에 Break point 설정/해제
**ctrl + alt + R / ctrl + f11** : 실행
**f11** : 디버깅 시작
**f5** : step into / **f6** : step over
**f8** : 디버깅 계속
**ctrl + .** : 다음오류부분으로 가기
**ctrl + ,** : 이전오류부분으로 가기
**f12** : 에디터로 커서이동
**ALT + UP,DOWN** : 현재 줄 위치 이동
**Ctrl + j** : 검색할 단어를 입력하면서 실시간으로 검색
**Ctrl + Shift + j** : 검색할 단어를 입력하면서 실시간으로 거꾸로 검색
**F4** : 클래스명을 선택하고 누르면 해당 클래스의 Hierarchy 를 볼 수 있다.
**ctrl + alt + up/down** : 한줄 duplicate
**alt + shift + 방향** : 선택
**ctrl + shift + g** : 캐럿이 위치한 객체가 참조 되는 곳을 찾아 준다

# 4 템플릿 사용단축키

sysout 입력한 후 Ctrl + Space 하면 System.out.println(); 으로 바뀐다.
try 입력한 후 Ctrl + Space 하면 try-catch 문이 완성된다.
for 입력한 후 Ctrl + Space 하면 여러가지 for 문을 완성할 수 있다.
템플릿을 수정하거나 추가하려면 환경설정/자바/편집기/템플리트 에서 할 수 있다.

# 5 편집기 변환

에디터가 여러 파일을 열어서 작업중일때 Ctrl + F6 키를 누르면 여러파일명이 나오고 F6 키를 계속 누르면 아래로
Ctrl + Shift + F6 키를 누르면 위로 커서가 움직인다.
Ctrl + F7 : 뷰간 전환
Ctrl + F8 : 퍼스펙티브간 전환
F12 : 에디터로 포커스 위치